

KIẾN TRÚC PHẦN MỀM

SOFTWARE ARCHITECTURE

- ThS. Đoàn Chánh Thống
- ThS. Nguyễn Cường Phát
- ThS. Nguyễn Hữu Lợi
- ThS. Trương Quốc Dũng
- ThS. Nguyễn Thành Hiệp
- ThS. Võ Duy Nguyên
- ThS. Nguyễn Văn Toàn
- ThS. Lê Ngô Thục Vi
- TS. Nguyễn Duy Khánh
- TS. Nguyễn Tấn Trần Minh Khang

What is Software Architecture?

KIẾN TRÚC PHẦN MỀM LÀ GÌ?

What is Software Architecture?

- Software architecture is the overall structure of a software system, defining how its components interact, how data flows, and the design principles that ensure the system is efficient, maintainable, and scalable.
- Kiến trúc phần mềm (Software Architecture) là cấu trúc tổng thể của một hệ thống phần mềm, kiến trúc phần mềm bao gồm các thành phần trong hệ thống, cách các thành phần tương tác với nhau, cách dữ liệu luân chuyển và nguyên tắc thiết kế để đảm bảo hệ thống hoạt động hiệu quả, dễ bảo trì và mở rộng.

Main Components of SA

CÁC THÀNH PHẦN CHÍNH CỦA KIẾN TRÚC PHẦN MỀM

Main Components of SA

- Components:
 - + These are modules, classes, or services within the system.
 - + Each component has a specific function or responsibility.
- Interactions & Dependencies:
 - + Defines how components communicate and exchange data.
 - + Example: Communication between Frontend and Backend, or between microservices.
- Design Principles:
 - + Separation of Concerns: Divides the system into independent parts with distinct responsibilities.
 - + Scalability: Ensures the system can handle increased user demand.
 - + Security: Protects the system from potential threats.
 - + Maintainability: Makes it easier to modify and upgrade the system over time.

Các thành phần chính của SA

- Các thành phần (Components):
 - + Các thành phần là các mô-đun, các lớp đối tượng, hoặc các dịch vụ trong hệ thống.
 - + Mỗi thành phần có một chức năng riêng biệt.
- Mỗi quan hệ giữa các thành phần (Interactions & Dependencies):
 - + Chỉ ra cách các thành phần giao tiếp và trao đổi dữ liệu.
 - + Ví dụ: Giao tiếp giữa Frontend và Backend, hoặc giữa các dịch vụ microservices.
- Nguyên tắc thiết kế (Design Principles):
 - + Separation of Concerns (Phân tách): Chia hệ thống thành các phần có trách nhiệm riêng biệt.
 - + Scalability (Khả năng mở rộng): Đảm bảo hệ thống có thể mở rộng khi số lượng người dùng tăng lên.
 - + Security (Bảo mật): Đảm bảo hệ thống an toàn trước các mối đe dọa.
 - + Maintainability (Dễ bảo trì): Giúp hệ thống dễ sửa đổi và nâng cấp trong tương lai.

Common Software Architecture Patterns

CÁC MÔ HÌNH KIẾN TRÚC PHẦN MỀM PHỔ BIẾN

Common Software Architecture Patterns

1. Monolithic Architecture – Kiến trúc nguyên khối.
2. Client-Server Architecture – Kiến trúc khách-chủ.
3. Layered Architecture – Kiến trúc phân lớp.
4. MVC (Model-View-Controller) Architecture – Kiến trúc MVC.
5. Event-Driven Architecture – Kiến trúc hướng sự kiện.
6. Microservices Architecture – Kiến trúc hướng dịch vụ.
7. Serverless Architecture – Kiến trúc không máy chủ.

Các mô hình kiến trúc phần mềm

- **Monolithic Architecture** is a software design where the entire application is built as a single, unified unit, with all components (UI, business logic, and database) tightly integrated into one system.
- **Kiến trúc đơn khối (kiến trúc nguyên khối)** là một thiết kế phần mềm trong đó toàn bộ ứng dụng được xây dựng như một khối thống nhất, với tất cả các thành phần (giao diện người dùng, logic nghiệp vụ và cơ sở dữ liệu) được tích hợp chặt chẽ vào một hệ thống duy nhất.

Các mô hình kiến trúc phần mềm

- Client-Server Architecture: The system consists of a server handling requests from multiple clients.
- Kiến trúc Client-Server: Gồm máy chủ (Server) xử lý yêu cầu từ máy khách (Client).

Các mô hình kiến trúc phần mềm

- **Layered Architecture**: Divides the system into layers like Presentation (UI), Business Logic, and Data Access.
- **Kiến trúc phân tầng**: Chia hệ thống phần mềm thành nhiều tầng như Presentation (tầng Giao diện), Business Logic (tầng Xử lý nghiệp vụ), Data Access (tầng Truy cập dữ liệu).

Các mô hình kiến trúc phần mềm

- MVC (Model-View-Controller): Separates data (Model), UI (View), and logic (Controller).
- Kiến trúc MVC (Model-View-Controller): Tách biệt giữa dữ liệu (Model), giao diện (View) và điều khiển (Controller).

Các mô hình kiến trúc phần mềm

- **Event-Driven Architecture**: Components communicate via events instead of direct calls.
- **Kiến trúc hướng sự kiện**: Các thành phần giao tiếp qua sự kiện (event).

Các mô hình kiến trúc phần mềm

- **Microservices Architecture**: Breaks the system into small, independent services.
- **Kiến trúc hướng dịch vụ**: Chia nhỏ hệ thống thành nhiều dịch vụ nhỏ, độc lập.

Các mô hình kiến trúc phần mềm

- **Serverless Architecture** is a cloud-based computing model where developers focus on writing code without managing servers, with the cloud provider automatically handling the infrastructure and scaling needs.
- **Kiến trúc không máy chủ** là mô hình điện toán đám mây cho phép nhà phát triển chỉ tập trung vào việc viết mã mà không cần quản lý máy chủ và nhà cung cấp đám mây tự động xử lý cơ sở hạ tầng và yêu cầu mở rộng.

Why is SA Important?

**TẠI SAO KIẾN TRÚC PHẦN MỀM QUAN
TRỌNG**

Why is SA Important?

- Ensures scalability and maintainability.
- Improves performance and code reusability.
- Enhances security and system reliability.
- Helps development teams work efficiently with a clear structure.

Software architecture is a crucial aspect of software development, enabling the creation of robust, flexible, and scalable systems.

Tại sao kiến trúc phần mềm quan trọng

- Giúp hệ thống dễ mở rộng và bảo trì.
- Cải thiện hiệu suất và khả năng tái sử dụng mã nguồn.
- Đảm bảo bảo mật và khả năng chịu lỗi của hệ thống.
- Giúp các nhóm phát triển làm việc hiệu quả hơn với quy trình rõ ràng.

Kiến trúc phần mềm là một phần quan trọng trong phát triển phần mềm, giúp xây dựng các hệ thống mạnh mẽ, linh hoạt và có khả năng mở rộng cao.

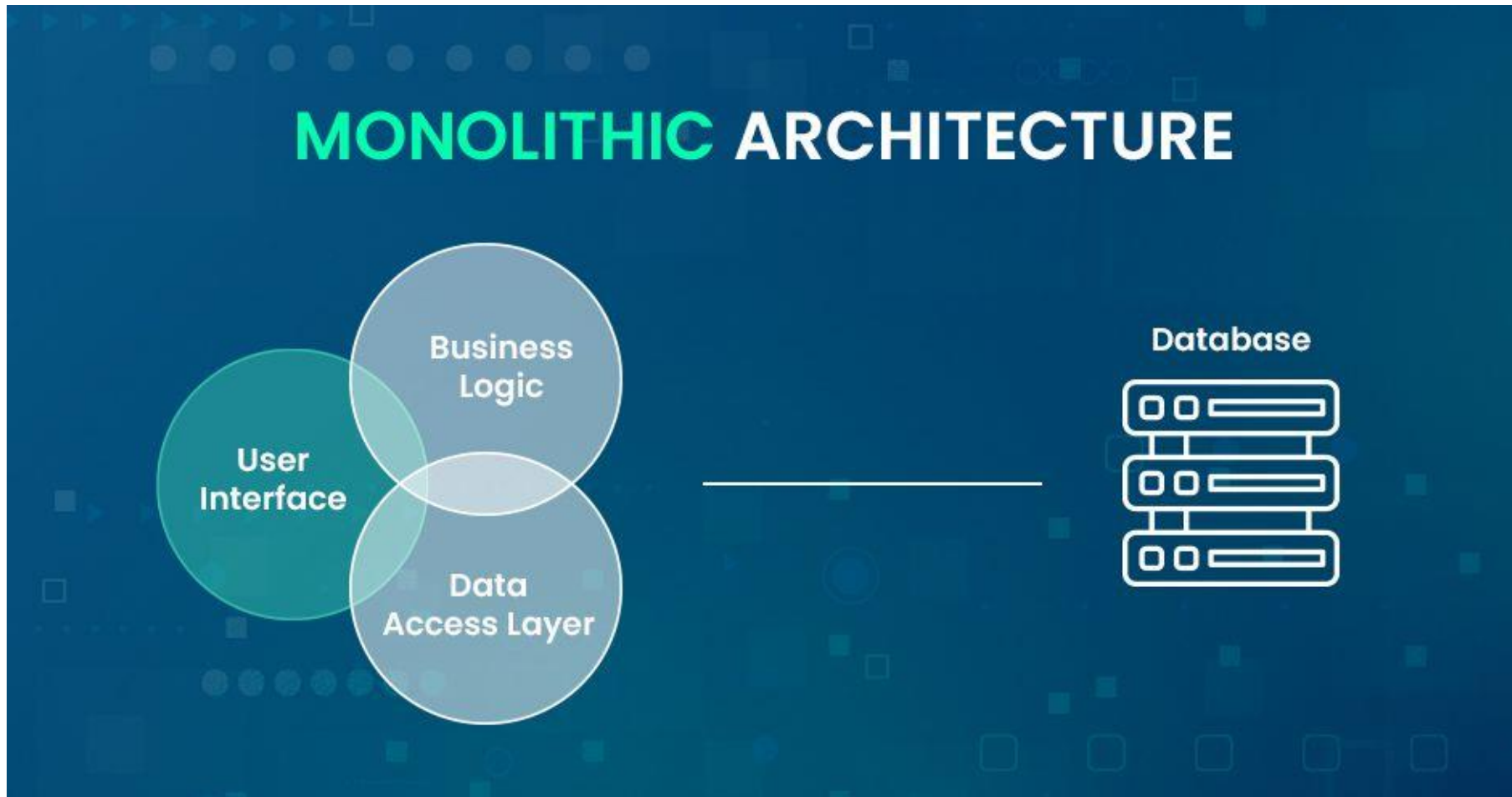
Common Software Architecture Patterns

1. Monolithic Architecture.
2. Client-Server Architecture.
3. Layered Architecture.
4. MVC (Model-View-Controller) Architecture.
5. Event-Driven Architecture.
6. Microservices Architecture.
7. Serverless Architecture.

Monolithic Architecture.

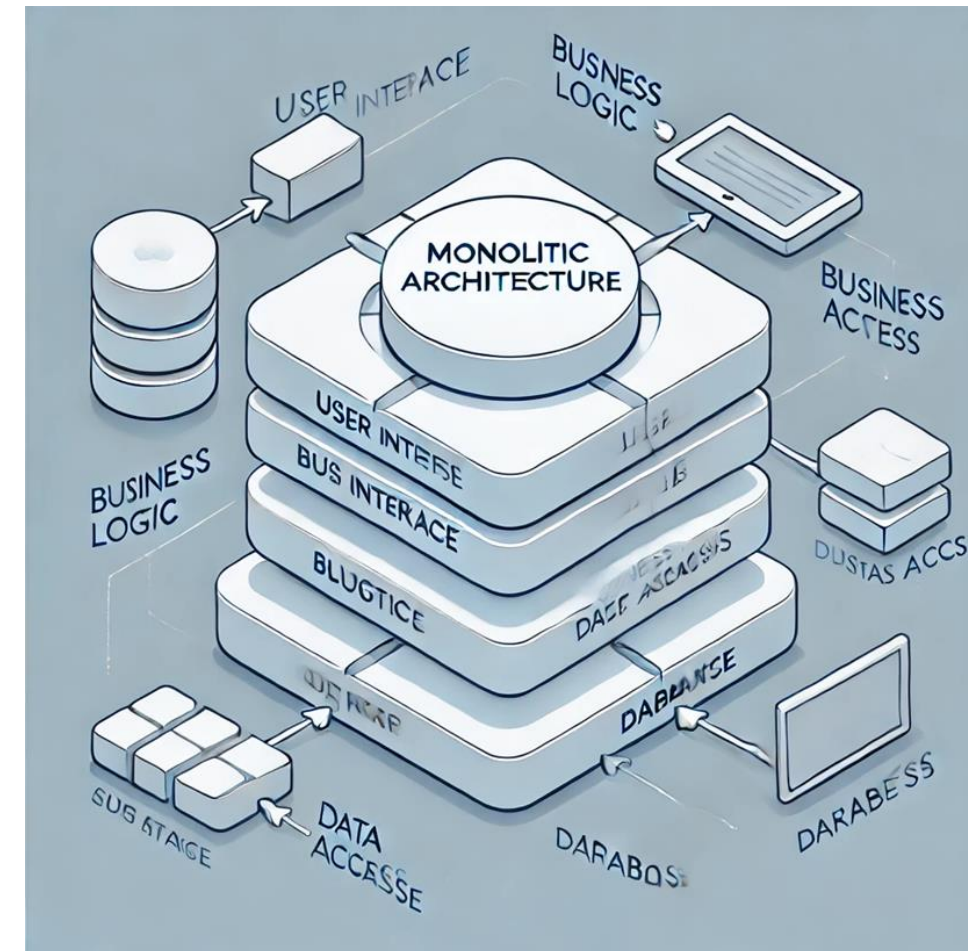
KIẾN TRÚC NGUYÊN KHỐI
KIẾN TRÚC ĐƠN KHỐI

Monolithic Architecture



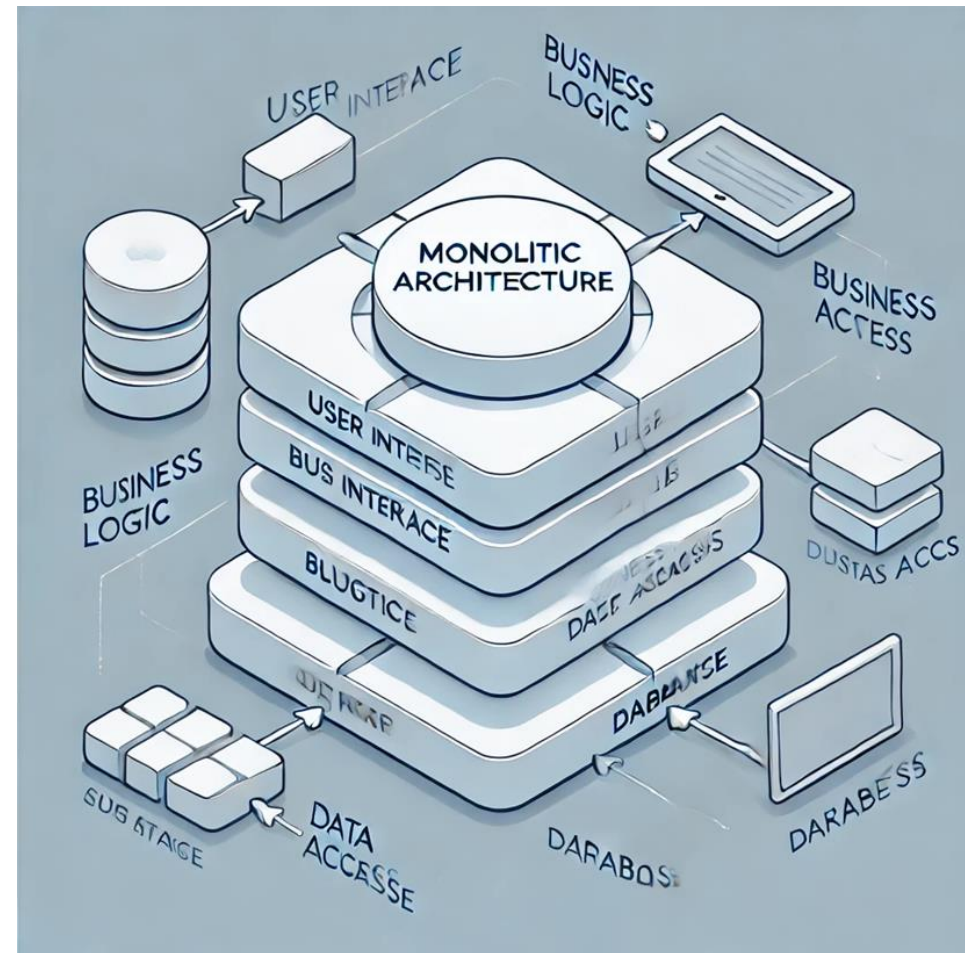
Monolithic Architecture

- Monolithic Architecture is a software architecture model where the entire application is built, deployed, and runs as a single unit. All components of the system (UI, business logic, and database) are integrated into one unified codebase.



Kiến trúc nguyên khối

- Kiến trúc nguyên khối (Monolithic Architecture) là một mô hình kiến trúc phần mềm trong đó toàn bộ ứng dụng được phát triển, triển khai và chạy như một khối duy nhất. Tất cả các thành phần của hệ thống (giao diện, logic xử lý, cơ sở dữ liệu) được tích hợp trong một ứng dụng duy nhất.



Structure of Monolithic Architecture

- A Monolithic application typically consists of:
 1. User Interface (UI) – The frontend that users interact with.
 2. Business Logic Layer – Handles the application's core logic and processes.
 3. Data Access Layer – Manages database interactions.
 4. Database – Stores all application data.
- All these components run together in a single application, hosted on one server or a cluster of servers.

Cấu trúc của MA

- Một ứng dụng nguyên khối thường bao gồm:
 1. User Interface (UI) – Thành phần Giao diện người dùng (Frontend).
 2. Business Logic Layer – Thành phần Xử lý logic nghiệp vụ.
 3. Data Access Layer – Thành phần Truy xuất và quản lý dữ liệu.
 4. Database – Thành phần Cơ sở dữ liệu chứa toàn bộ thông tin của hệ thống.
- Tất cả các thành phần này nằm trong một ứng dụng duy nhất, chạy trên một server hoặc một cụm máy chủ.

How Monolithic Architecture Works

1. The user sends a request through the UI.
2. The request is processed in the Business Logic Layer.
3. The system retrieves data from the Database Layer.
4. The response is sent back to the user via the UI.

Cách hoạt động của MA

1. Người dùng gửi yêu cầu đến hệ thống (qua thành phần UI).
2. Hệ thống xử lý yêu cầu trong thành phần Business Logic Layer.
3. Hệ thống truy xuất dữ liệu từ thành phần Database Layer.
4. Trả về kết quả cho người dùng qua thành phần UI.

Advantages of Monolithic Architecture

1. Easy to develop initially – Simple codebase and deployment.
2. High performance for small applications – No inter-service communication overhead.
3. Easier testing and debugging – Everything is in a single project.
4. Simpler management – Only one application needs to be deployed.

Ưu điểm của MA

- Dễ phát triển ban đầu – Codebase đơn giản, dễ triển khai.
- Hiệu suất cao trong môi trường nhỏ – Không có độ trễ giữa các dịch vụ.
- Dễ kiểm thử (Testing) và gỡ lỗi (Debugging) – Vì tất cả các thành phần nằm trong một codebase duy nhất.
- Quản lý đơn giản – Chỉ cần triển khai một ứng dụng thay vì nhiều dịch vụ riêng biệt.

Disadvantages of MA

- Difficult to scale – As the system grows, it becomes harder to expand efficiently.
- Hard to maintain and update – Even small changes require rebuilding and redeploying the entire system.
- Low fault tolerance – A failure in one module can crash the entire system.
- Limited technology flexibility – All components must use the same technology stack.

Nhược điểm của MA

- Khó mở rộng – Khi hệ thống lớn lên, việc mở rộng từng phần trở nên khó khăn.
- Khó bảo trì và cập nhật – Một thay đổi nhỏ có thể ảnh hưởng toàn bộ hệ thống, cần build và deploy lại toàn bộ ứng dụng.
- Khả năng chịu lỗi kém – Nếu một thành phần bị lỗi, toàn bộ hệ thống có thể sập.
- Khó sử dụng công nghệ đa dạng – Mọi thành phần phải dùng chung một stack công nghệ.

Real-World Examples of MA

- Small business applications & startup MVPs
 - + Many startups begin with a Monolithic structure due to its simplicity and ease of deployment.
- Traditional enterprise applications
 - + Many older systems (e.g., ERP, internal management software) are built using Monolithic Architecture.
- Facebook's early architecture (2004-2009)
 - + Initially, Facebook was a Monolithic application but later transitioned to Microservices for better scalability.

Ví dụ thực tế về MA

- Ứng dụng doanh nghiệp nhỏ & startup ban đầu:
 - + Nhiều công ty khởi đầu với Monolithic Architecture vì dễ triển khai và quản lý.
- Ứng dụng web truyền thống:
 - + Các hệ thống cũ thường được xây dựng theo Monolithic (ERP, phần mềm quản lý doanh nghiệp).
- Facebook thời kỳ đầu (2004-2009):
 - + Facebook ban đầu là một ứng dụng Monolithic, nhưng sau đó phải chuyển sang Microservices để mở rộng.

When to Use Monolithic Architecture?

- When building a small application or MVP (Minimum Viable Product).
- When the development team is small, and the project doesn't need high scalability.
- When quick deployment and low initial cost are priorities.

Khi nào nên sử dụng MA

- Khi phát triển ứng dụng nhỏ hoặc MVP (Minimum Viable Product).
- Khi đội ngũ phát triển ít người và không muốn phức tạp hóa việc quản lý hệ thống.
- Khi muốn triển khai nhanh chóng và tiết kiệm chi phí ban đầu.

Conclusion

- Monolithic Architecture is ideal for small applications and early-stage startups due to its simplicity and easy management.
- However, as applications grow, Microservices Architecture becomes a better choice for scalability, flexibility, and maintainability.

Kết luận

- Kiến trúc nguyên khối phù hợp cho các ứng dụng nhỏ hoặc startup cần phát triển nhanh.
- Tuy nhiên, khi hệ thống phát triển lớn hơn, việc chuyển sang Kiến trúc hướng dịch vụ (microservices architecture) sẽ giúp cải thiện khả năng mở rộng, bảo trì và hiệu suất.

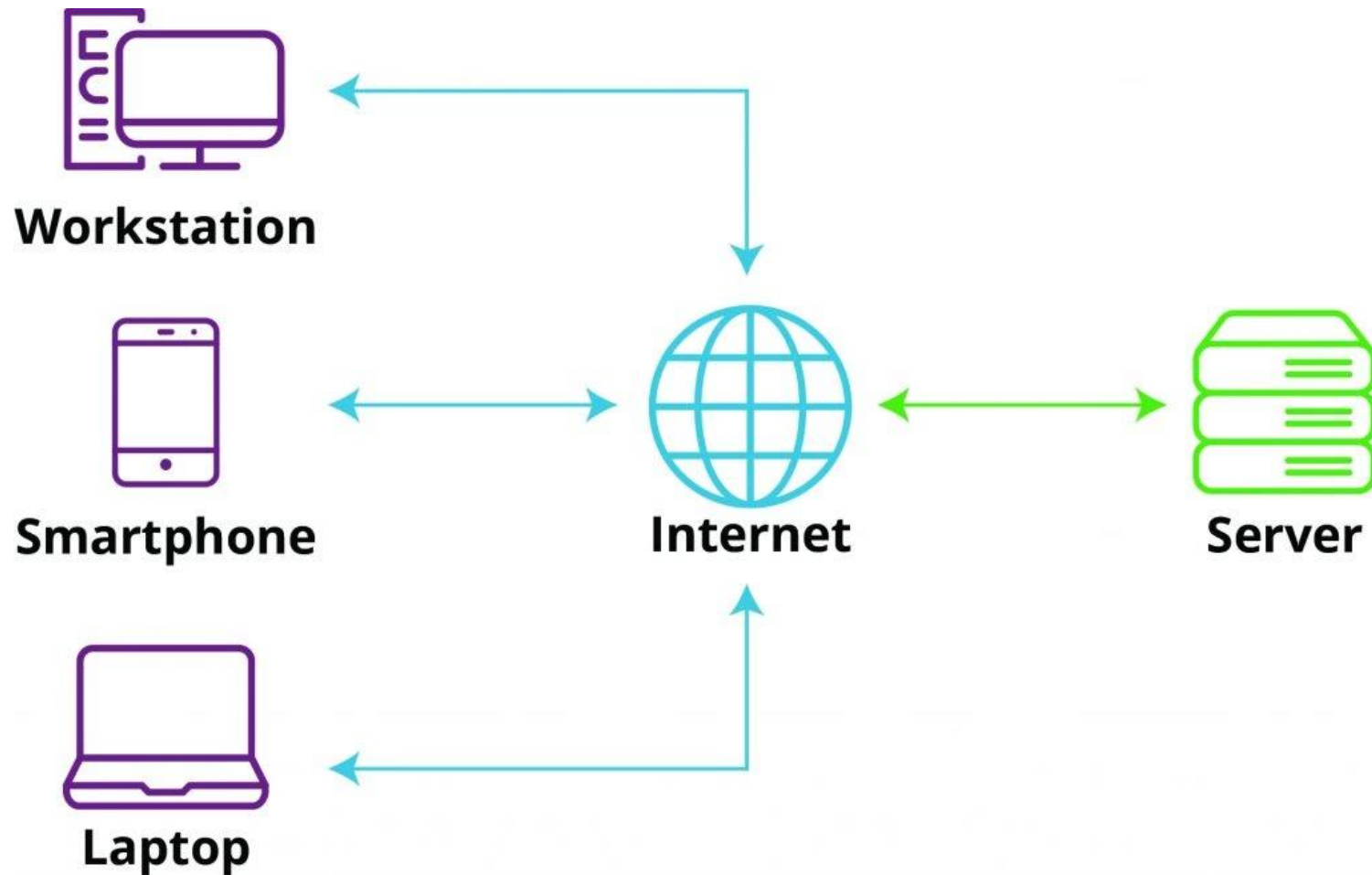
Common Software Architecture Patterns

1. Monolithic Architecture.
2. Client-Server Architecture.
3. Layered Architecture.
4. MVC (Model-View-Controller) Architecture.
5. Event-Driven Architecture.
6. Microservices Architecture.
7. Serverless Architecture.

Client-Server Architecture

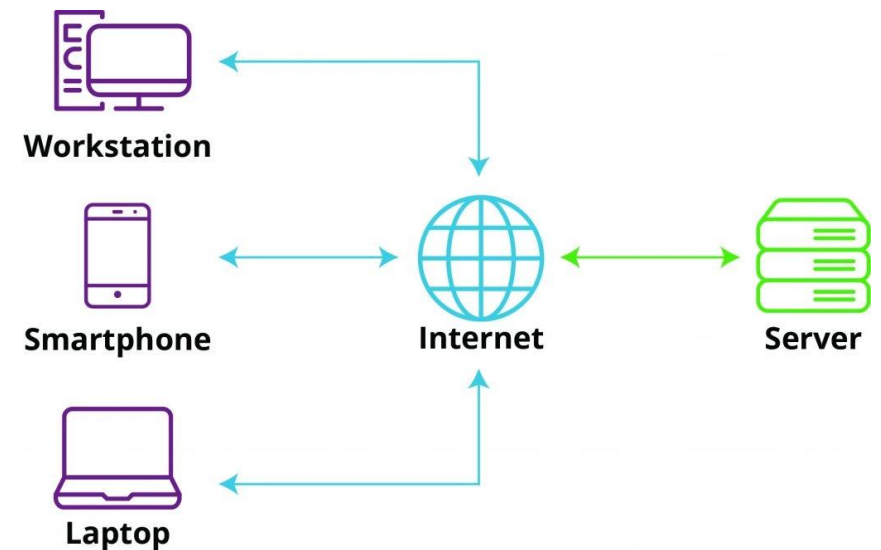
KIẾN TRÚC CLIENT-SERVER

Client-Server Architecture



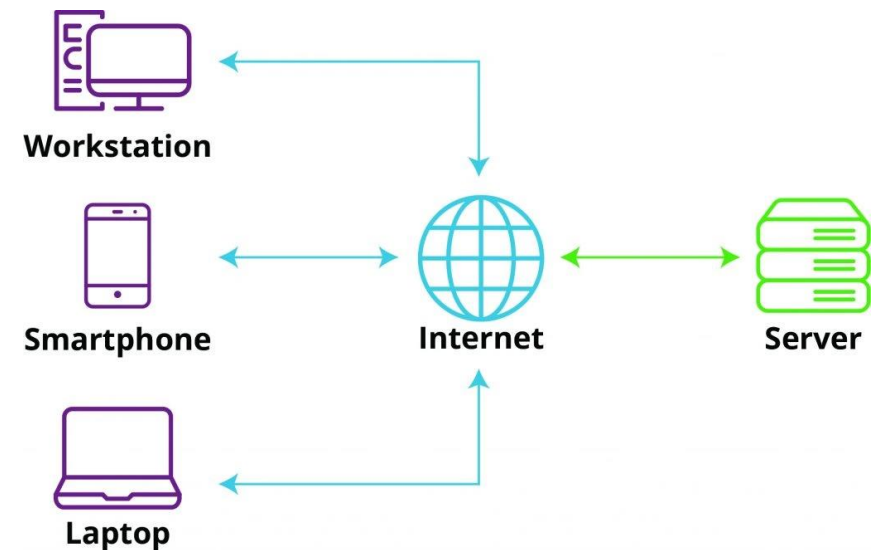
Client-Server Architecture

- Client-Server Architecture is a software architecture model where a Client (a user device or application) sends requests to a Server, which processes the request and returns the response.
- This architecture is widely used in web applications, mobile applications, and online services.



Kiến thức Client-Server Architecture

- Kiến trúc khách-chủ là một mô hình kiến trúc trong đó máy khách (client) gửi yêu cầu đến máy chủ (server) và máy chủ sẽ xử lý yêu cầu rồi trả kết quả về cho máy khách.
- Mô hình này được sử dụng rộng rãi trong các hệ thống mạng, ứng dụng web, ứng dụng di động và các dịch vụ trực tuyến.



Main Components of CSA

1. Client

- + A device or application that sends requests to the server.
- + Examples: Web browsers, mobile apps, desktop applications.
- + Example: When you access Facebook, your browser acts as the Client.

2. Server

- + A powerful system that processes requests from Clients and sends back responses.
- + Usually contains databases and application logic.
- + Example: When you log into Facebook, the Facebook Server checks your credentials and sends back your profile data.

3. Network

- + Connects Clients and Servers, allowing communication via the Internet or a private network (LAN, WAN).

Thành phần chính của CSA

1. Client (Máy khách)

- + Máy khách là thiết bị hoặc ứng dụng gửi yêu cầu đến máy chủ.
- + Máy khách có thể là trình duyệt web, ứng dụng di động, ứng dụng desktop.
- + Ví dụ: Khi bạn mở trình duyệt và truy cập Facebook, trình duyệt là Client.

2. Server (Máy chủ)

- + Máy chủ xử lý các yêu cầu từ Client và trả về phản hồi.
- + Máy chủ thường chứa cơ sở dữ liệu (database) và các tài nguyên cần thiết.
- + Ví dụ: Khi bạn đăng nhập vào Facebook, máy chủ Facebook kiểm tra tài khoản và trả về dữ liệu của bạn.

3. Network (Mạng)

- + Mạng là phương tiện giúp Client và Server kết nối với nhau, có thể là Internet hoặc mạng nội bộ (LAN, WAN).

How Client-Server Architecture Works

1. Client sends a request to the Server (e.g., login request).
2. Server processes the request and retrieves data.
3. Server sends a response back to the Client.
4. Client displays the data to the user (e.g., showing the user profile).

Cách hoạt động của CSA

1. Client gửi yêu cầu đến Server
 - + Ví dụ: yêu cầu đăng nhập.
2. Server nhận yêu cầu, xử lý và truy vấn dữ liệu.
3. Server trả kết quả về cho Client.
4. Client hiển thị dữ liệu cho người dùng.
 - + Ví dụ: giao diện hiển thị thông tin tài khoản.

Common Types of CSA

1. 2-Tier Architecture

- + The Client communicates directly with the Server.
- + The Server processes requests and retrieves data from the database.
- + Example: A direct connection between a mobile app and a MySQL database.

2. 3-Tier Architecture

- + Includes Client, Application Server, and Database Server.
- + The Application Server processes logic before retrieving data from the Database Server.
- + Example: A web application with Frontend (Client) - Backend (Server) - Database (Data Server).

3. Multi-Tier Architecture

- + Expands 3-Tier Architecture by adding Load Balancers, API Gateways, and Caching Servers.
- + Used in large-scale systems like Amazon, Google, and Netflix.

Các loại CSA phổ biến

1. **Kiến trúc 2 tầng (2-Tier Architecture)**
 - + Client giao tiếp trực tiếp với Server.
 - + Dữ liệu được xử lý tại Server và gửi kết quả cho Client.
 - + Ví dụ: Ứng dụng truy cập cơ sở dữ liệu trực tiếp (MySQL, PostgreSQL).
2. **Kiến trúc 3 tầng (3-Tier Architecture)**
 - + Gồm 3 thành phần: Client, Server ứng dụng (Application Server) và Database Server.
 - + Tầng Application Server trung gian xử lý logic, giúp tăng bảo mật và hiệu suất.
 - + Ví dụ: Ứng dụng web có Frontend (Client) - Backend (Server) - Database (Data Server).
3. **Kiến trúc đa tầng (Multi-Tier Architecture)**
 - + Mở rộng 3-tier bằng cách thêm các tầng dịch vụ khác như Load Balancer, API Gateway, Caching Server.
 - + Kiến trúc đa tầng được sử dụng trong các hệ thống lớn như Amazon, Google, Netflix.

Advantages of CSA

1. Optimized resource usage – Processing is handled by the Server, reducing Client workload.
2. Easy to maintain and scale – Servers can be upgraded without affecting Clients.
3. Improved security – Centralized control over data access.
4. Better scalability – Can handle multiple Clients efficiently.

Ưu điểm của CSA

1. Tối ưu tài nguyên – Xử lý dữ liệu tập trung trên Server giúp Client không cần phải thực hiện các tác vụ phức tạp.
2. Dễ bảo trì và mở rộng – Có thể nâng cấp Server mà không ảnh hưởng đến Client.
3. Bảo mật cao hơn – Dữ liệu được kiểm soát tập trung, giảm nguy cơ mất mát.
4. Khả năng mở rộng tốt – Dễ dàng thêm nhiều Client mà không làm quá tải hệ thống.

Disadvantages of CSA

1. Server dependency – If the Server fails, all Clients are affected.
2. Possible overload issues – High traffic can cause performance bottlenecks.
3. Higher cost – Requires investment in high-performance Servers.

Nhược điểm của CSA

1. Phụ thuộc vào Server – Nếu Server gặp lỗi, tất cả Client sẽ bị ảnh hưởng.
2. Có thể gây quá tải – Nếu số lượng Client tăng quá nhanh, Server có thể bị quá tải.
3. Chi phí cao – Cần đầu tư vào Server mạnh để đảm bảo hiệu suất cao.

Real-World Examples of CSA

1. Facebook, Google, YouTube – Web apps serving content from a central Server.
2. Online banking applications – A mobile banking app requests account balance from a Bank Server.
3. Online gaming – A Client (player) sends game actions to the Server, which updates the game state.

Ví dụ thực tế về CSA

1. Facebook, Google, YouTube – Các ứng dụng web sử dụng kiến trúc Client-Server để phục vụ nội dung từ máy chủ.
2. Ứng dụng ngân hàng trực tuyến – Ứng dụng mobile gửi yêu cầu kiểm tra số dư tài khoản lên Server.
3. Game online – Client (người chơi) gửi yêu cầu đến Server để cập nhật trạng thái trò chơi.

When to Use CSA?

- When building systems that require remote access by multiple users.
- When centralizing data management for security and control.
- When scalability is needed to support growing user demands.

Khi nào nên sử dụng CSA?

- Khi cần xây dựng hệ thống có nhiều người dùng truy cập từ xa.
- Khi cần kiểm soát tập trung dữ liệu để tăng tính bảo mật.
- Khi cần cấu trúc linh hoạt, có thể mở rộng để phục vụ nhiều Client.

Conclusion Client-Server Architecture

- Client-Server Architecture is a widely used design where a Client sends requests, and a Server processes and responds.
- It improves efficiency, security, and scalability but requires strong infrastructure to prevent performance issues.

Kết luận CSA

- Kiến trúc Client-Server là một kiến trúc phổ biến giúp tổ chức hệ thống theo mô hình Client gửi yêu cầu – Server xử lý và phản hồi.
- Kiến trúc Client-Server giúp tối ưu hiệu suất, tăng bảo mật và dễ dàng mở rộng, nhưng cũng yêu cầu hạ tầng mạnh mẽ để tránh tình trạng quá tải.

Common Software Architecture Patterns

1. Monolithic Architecture.
2. Client-Server Architecture.
3. Layered Architecture.
4. MVC (Model-View-Controller) Architecture.
5. Event-Driven Architecture.
6. Microservices Architecture.
7. Serverless Architecture.

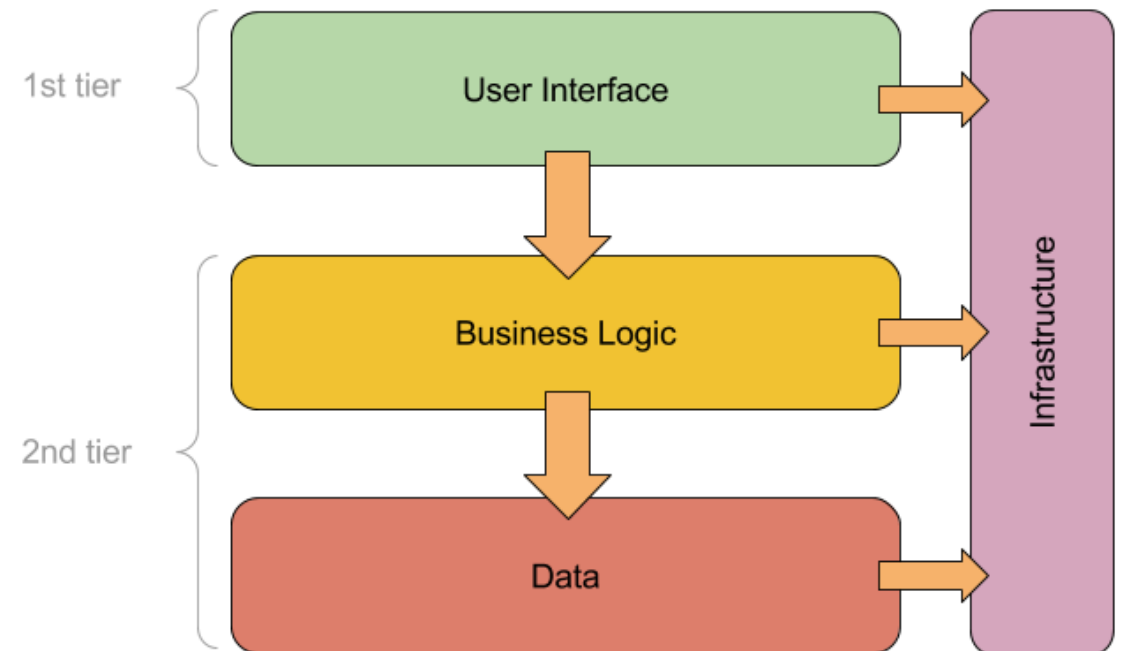
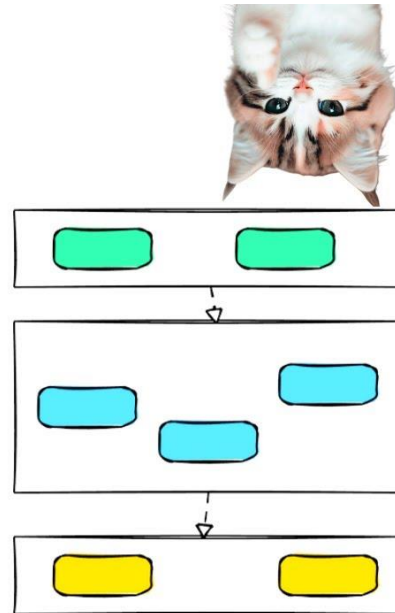
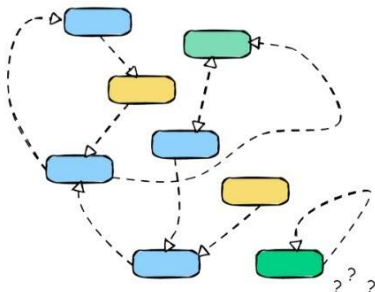
Layered Architecture

KIẾN TRÚC PHÂN TẦNG

KIẾN TRÚC PHÂN LỚP

Layered Architecture

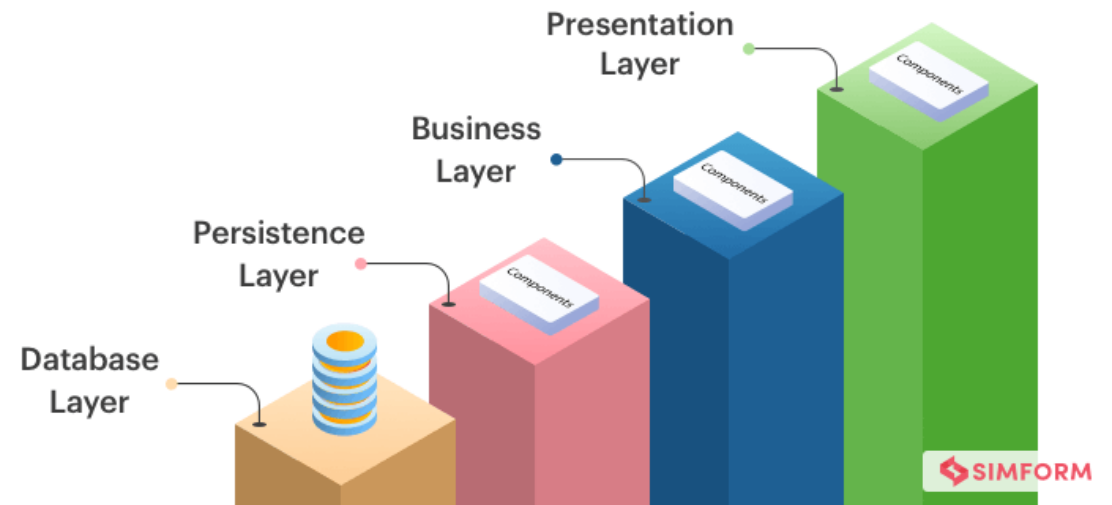
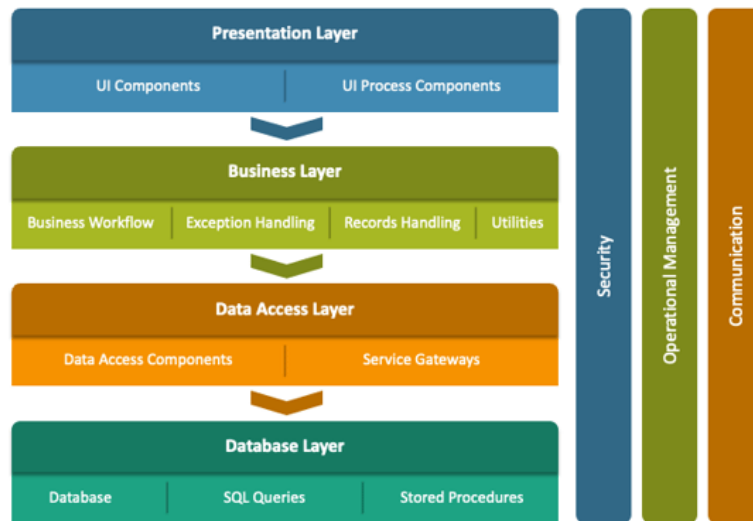
3 Layer Architecture



www.herbertograca.com

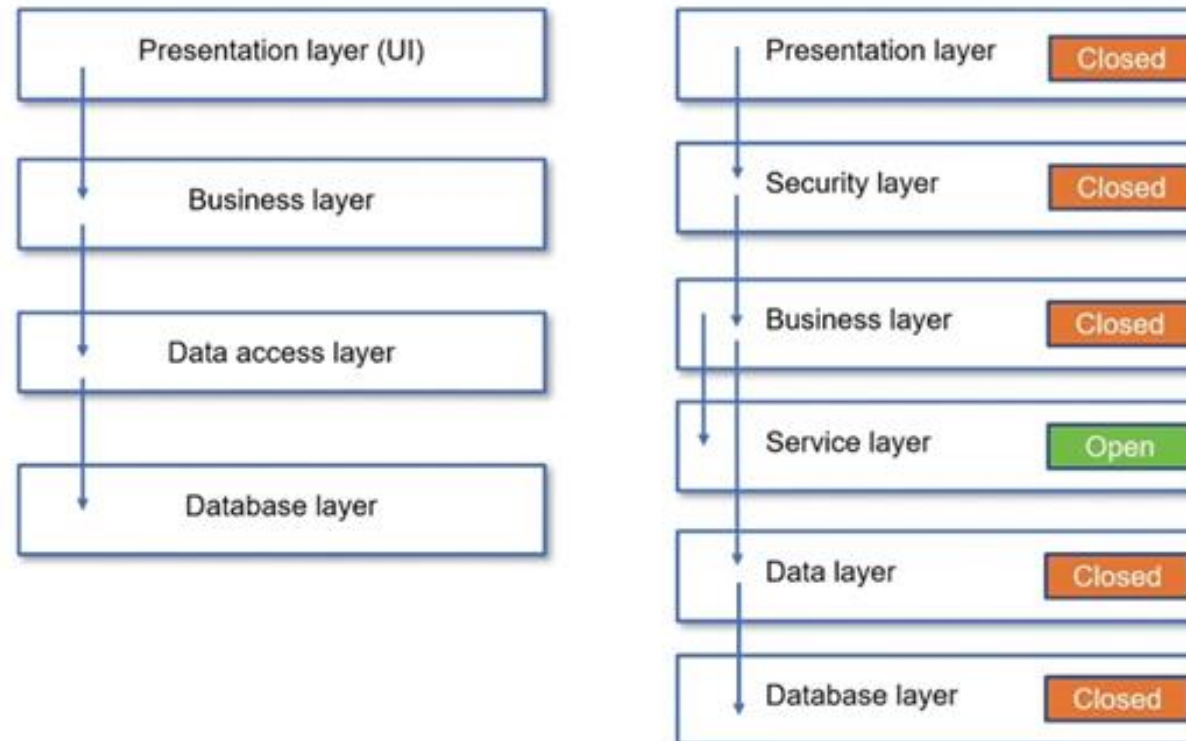
Layered Architecture

N-TIER ARCHITECTURE

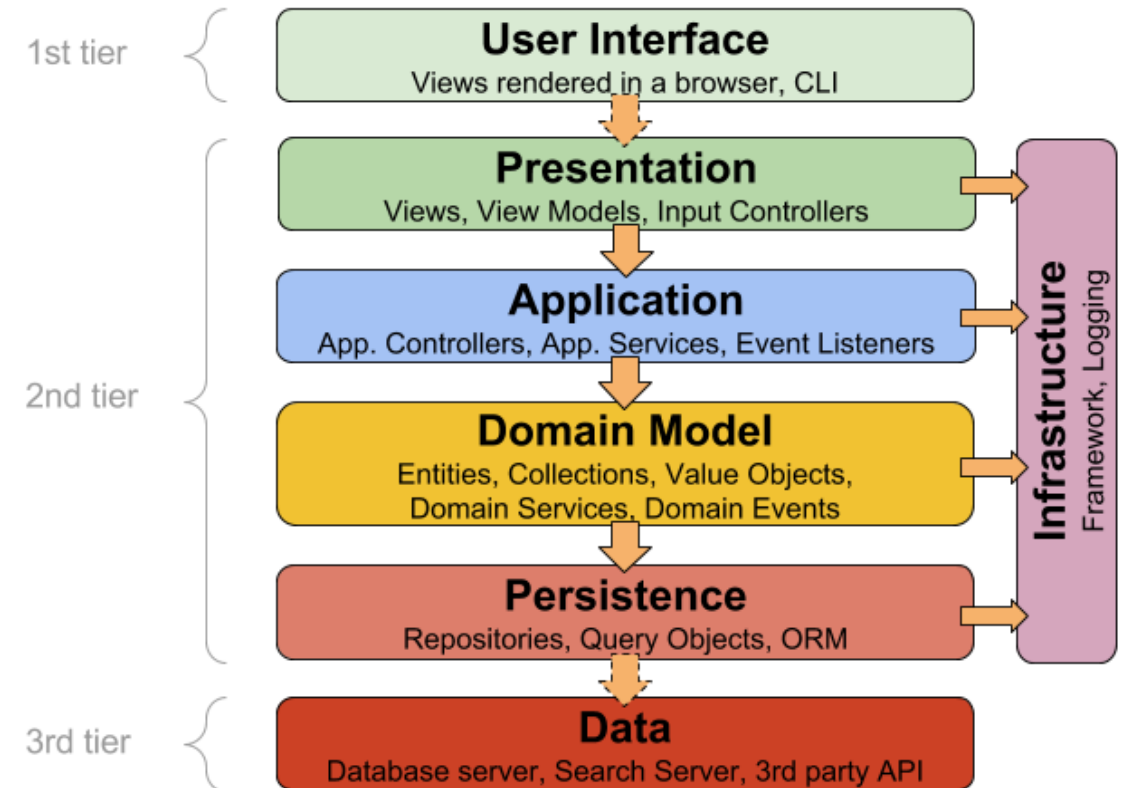
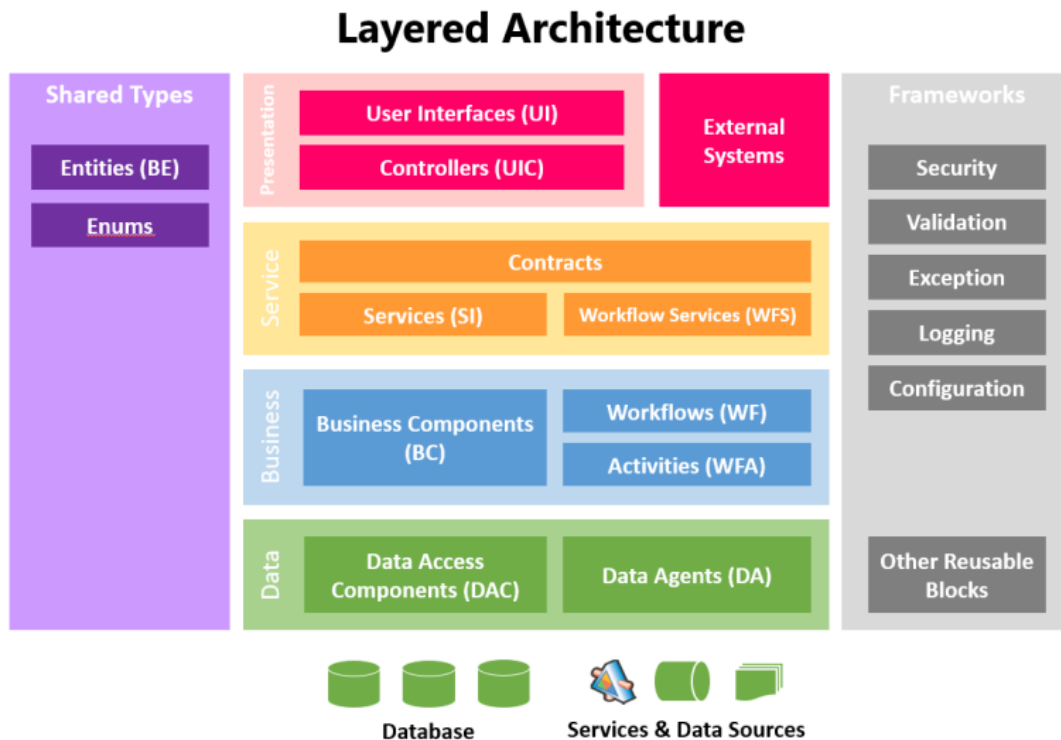


Layered Architecture

The Layered architecture



Layered Architecture



www.herbertograca.com

Layered Architecture

- Layered Architecture is a software design pattern where the system is divided into separate layers, each with a specific responsibility, and communication occurs only between adjacent layers.
- This architecture improves code organization, maintainability, reusability, and scalability.

Kiến trúc phân tầng

- Kiến trúc phân tầng là một mô hình thiết kế phần mềm trong đó hệ thống được chia thành các tầng (layers) riêng biệt, mỗi lớp có một nhiệm vụ cụ thể và chỉ giao tiếp với tầng liền kề.
- Mô hình Kiến trúc phân tầng giúp tổ chức code rõ ràng, dễ bảo trì, tái sử dụng và mở rộng.

Common Layers in Layered Architecture

— Layered Architecture typically consists of four main layers:

1. **Presentation Layer (UI Layer)**

- + Provides the user interface (Web UI, Mobile UI).
- + Receives user input and sends it to the Business Layer.
- + Displays results processed by lower layers.

2. **Business Logic Layer (Service Layer)**

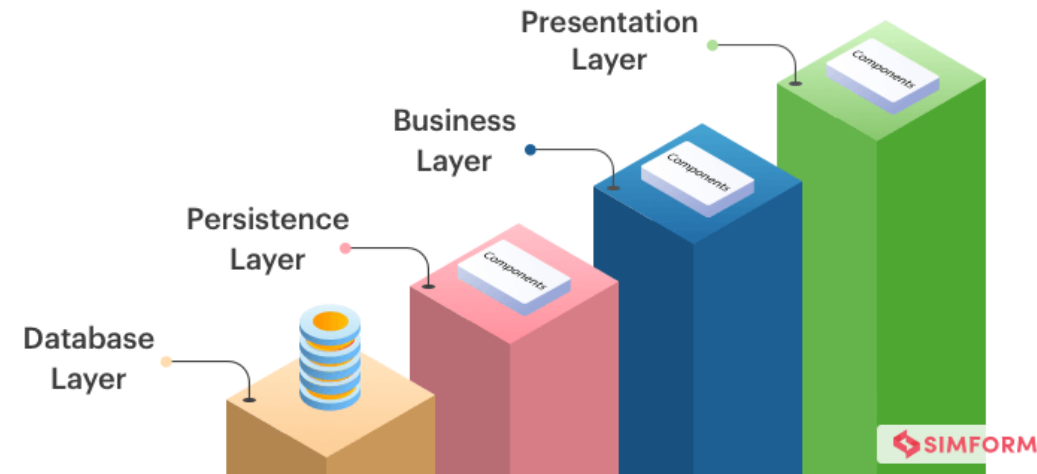
- + Contains the core business logic of the application.
- + Implements business rules, data validation, and processing.
- + Communicates with the Data Access Layer to fetch or modify data.

3. **Data Access Layer (Persistence Layer)**

- + Handles data retrieval, storage, and updates in the database.
- + Often uses ORM (Object-Relational Mapping) like Hibernate, Entity Framework.

4. **Database Layer**

- + Contains the actual database system (SQL Server, MySQL, MongoDB).
- + Stores all application data.



Các tầng trong kiến trúc phân tầng

— Kiến trúc phân lớp thường có 4 lớp chính:

1. **Presentation Layer (Tầng trình bày - UI Layer)**

- + Cung cấp giao diện người dùng (Web UI, Mobile UI).
- + Nhận dữ liệu đầu vào từ người dùng và gửi đến lớp Business.
- + Hiển thị kết quả sau khi xử lý từ lớp dưới.

2. **Business Logic Layer (Tầng nghiệp vụ)**

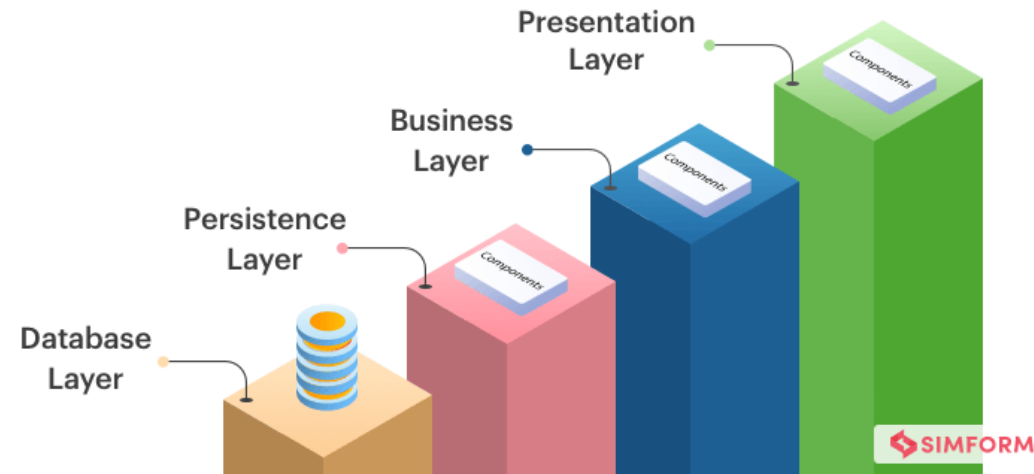
- + Chứa logic xử lý nghiệp vụ của ứng dụng.
- + Thực hiện các quy tắc tính toán, kiểm tra dữ liệu.
- + Giao tiếp với lớp Data Access để lấy dữ liệu.

3. **Data Access Layer (Tầng truy xuất dữ liệu)**

- + Chịu trách nhiệm truy vấn, lưu trữ, cập nhật dữ liệu trong cơ sở dữ liệu (Database).
- + Thường sử dụng ORM (Object-Relational Mapping) như Hibernate, Entity Framework.

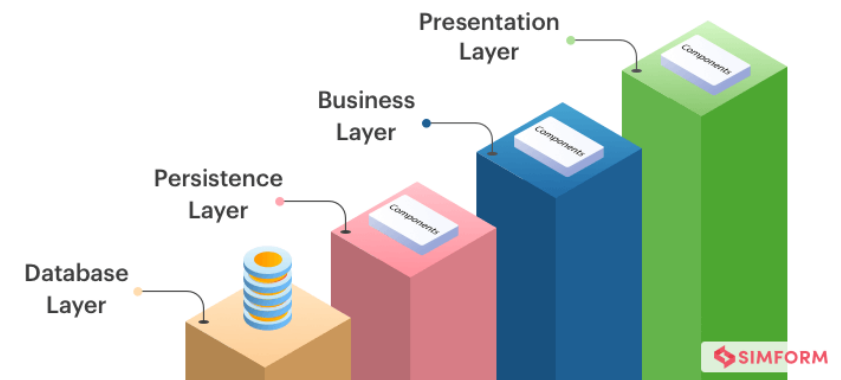
4. **Database Layer (Tầng cơ sở dữ liệu)**

- + Chứa hệ thống quản lý cơ sở dữ liệu (SQL Server, MySQL, MongoDB).
- + Lưu trữ dữ liệu của ứng dụng.



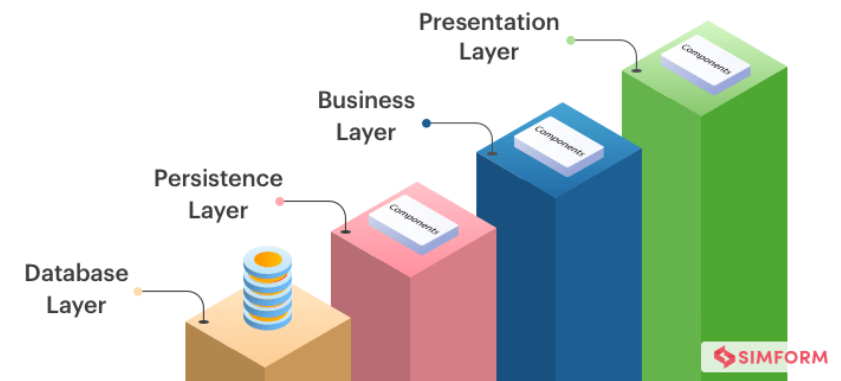
How Layered Architecture Works

1. The user interacts with the Presentation Layer (UI).
2. The Presentation Layer sends the request to the Business Logic Layer.
3. The Business Logic Layer processes the request and asks the Data Access Layer for data.
4. The Data Access Layer retrieves data from the Database Layer.
5. The data is returned in reverse order and finally displayed on the UI.



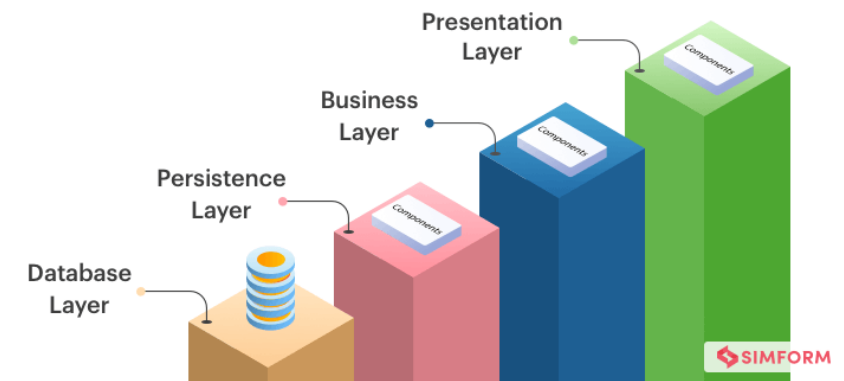
Hoạt động trong kiến trúc phân tầng

1. Người dùng tương tác với tầng giao diện (Presentation Layer).
2. Tầng Presentation gửi yêu cầu đến Tầng Business Logic.
3. Tầng Business Logic xử lý dữ liệu và yêu cầu Tầng Data Access lấy dữ liệu từ cơ sở dữ liệu.
4. Tầng Data Access truy vấn và gửi dữ liệu từ Tầng Database.
5. Dữ liệu được trả về theo thứ tự ngược lại và hiển thị trên tầng giao diện người dùng.



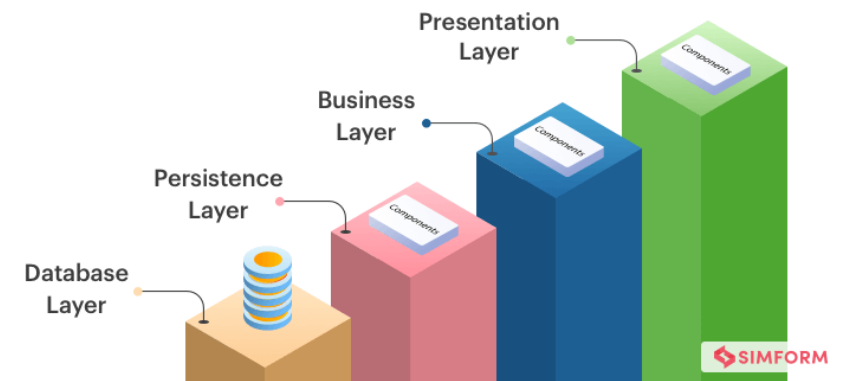
Advantages of Layered Architecture

1. Easy maintenance & scalability: Changes in one layer do not affect others.
2. High reusability: Layers can be reused in different applications.
3. Better data flow control: Ensures structured and organized data movement.
4. Improved security: Prevents direct access to the database.



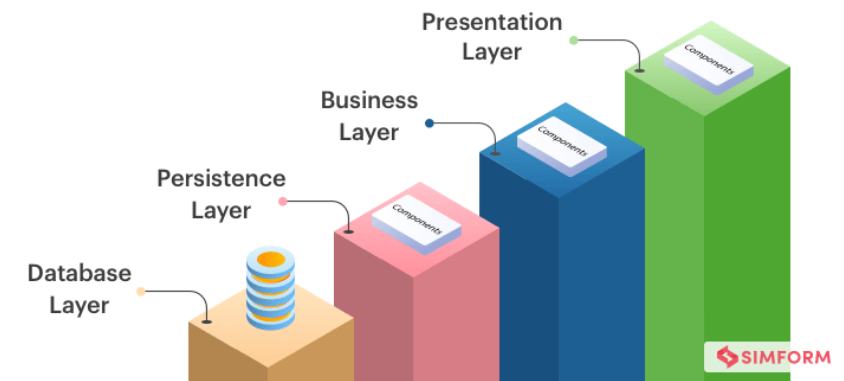
Ưu điểm của kiến trúc phân tầng

1. Dễ bảo trì & mở rộng: Chia tách rõ ràng các phần giúp dễ sửa đổi.
2. Tái sử dụng cao: Có thể sử dụng lại từng tầng trong các ứng dụng khác.
3. Kiểm soát luồng dữ liệu tốt hơn: Dữ liệu di chuyển có tổ chức giữa các tầng.
4. Bảo mật tốt hơn: Giới hạn truy cập trực tiếp vào tầng cơ sở dữ liệu.



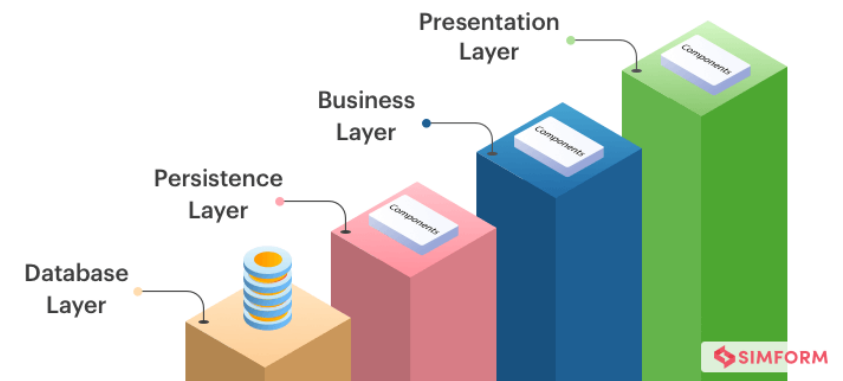
Disadvantages of Layered Architecture

1. Potential performance issues: Data must pass through multiple layers.
2. Less flexibility: Changes in one layer may require modifications in other layers.
3. Complex management for large applications: Keeping strict separation between layers can become challenging.



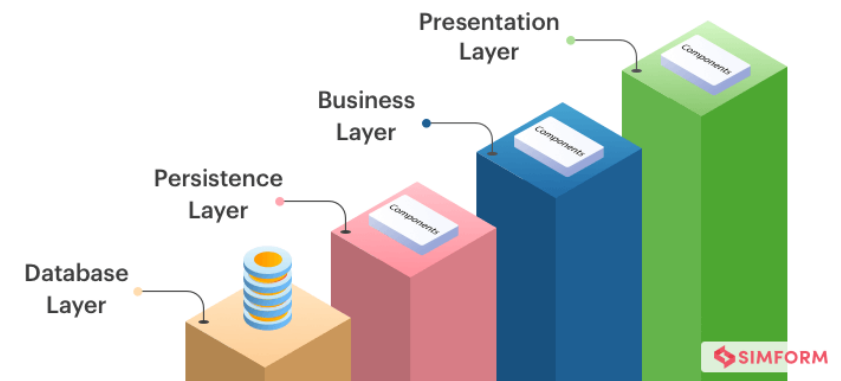
Nhược điểm của kiến trúc phân tầng

1. Hiệu suất có thể bị ảnh hưởng: Dữ liệu phải đi qua nhiều tầng trước khi đến đích.
2. Ít linh hoạt trong thay đổi: Nếu cần thay đổi một phần trong tầng Business, có thể phải cập nhật các tầng khác.
3. Khó quản lý khi mở rộng lớn: Khi hệ thống lớn hơn, việc duy trì sự tách biệt giữa các tầng có thể trở nên phức tạp.



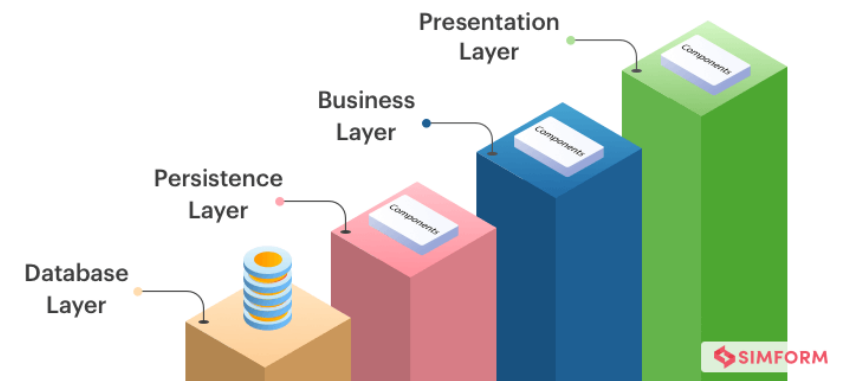
Example of Layered Architecture

- In an E-commerce application, the layers could be:
 - + **Presentation Layer:** Displays product listings.
 - + **Business Logic Layer:** Checks product availability.
 - + **Data Access Layer:** Fetches product data from the database.
 - + **Database Layer:** Stores product information.



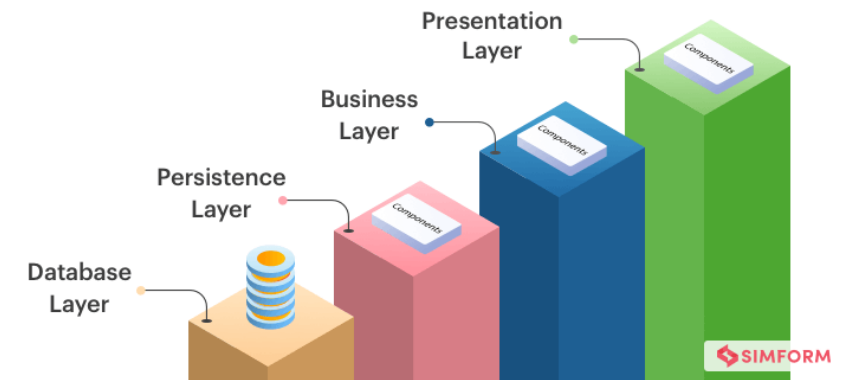
Ví dụ của kiến trúc phân tầng

- Một ứng dụng E-commerce có thể có các tầng như sau:
 - + Tầng Presentation: Trang web hiển thị danh sách sản phẩm.
 - + Tầng Business Logic: Kiểm tra xem sản phẩm còn hàng không.
 - + Tầng Data Access: Lấy dữ liệu sản phẩm từ database.
 - + Tầng Database: Cơ sở dữ liệu lưu trữ thông tin sản phẩm.



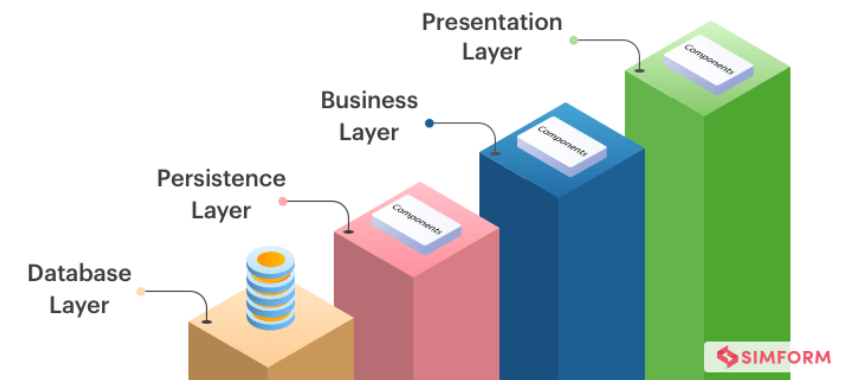
When to Use Layered Architecture?

- When developing medium to large-scale applications.
- When clear code organization and maintainability are required.
- When security and data control are essential.



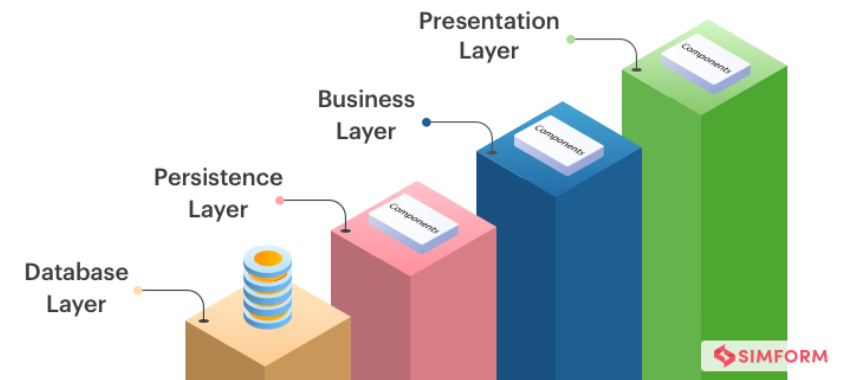
Khi nào sử dụng kiến trúc phân tầng

- Khi phát triển ứng dụng quy mô vừa và lớn.
- Khi cần tổ chức code rõ ràng và dễ bảo trì.
- Khi muốn đảm bảo tính bảo mật và kiểm soát tốt dữ liệu.



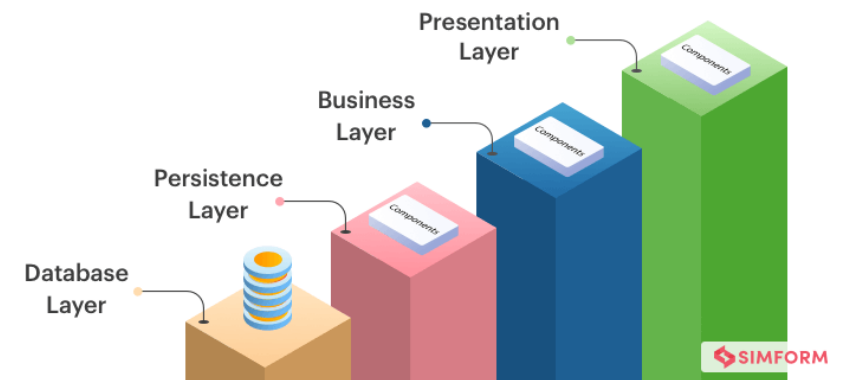
Conclusion Layered Architecture

- Layered Architecture is one of the most widely used software architectures, offering a well-structured and maintainable system.
- However, it can introduce performance overhead if not optimized properly.



Tổng kết của kiến trúc phân tầng

- Kiến trúc phân tầng là một trong những kiến trúc phổ biến nhất, giúp tổ chức hệ thống theo từng tầng rõ ràng, dễ bảo trì và nâng cấp.
- Tuy nhiên, kiến trúc phân tầng cũng có thể làm giảm hiệu suất nếu không được tối ưu hợp lý.



Common Software Architecture Patterns

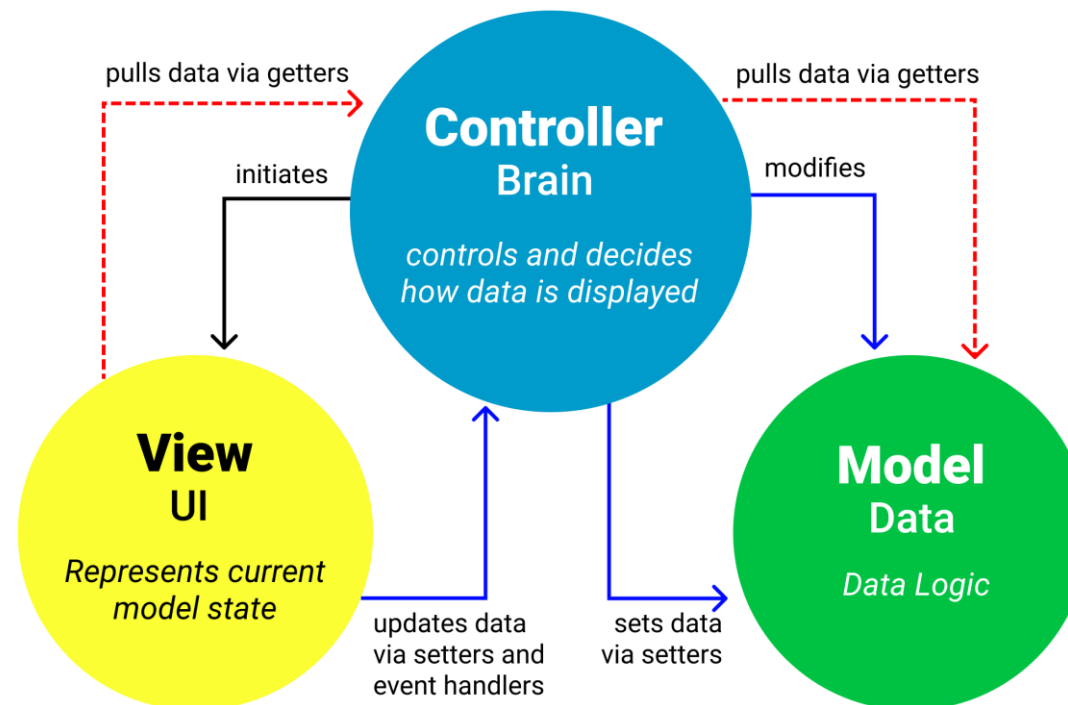
1. Monolithic Architecture.
2. Client-Server Architecture.
3. Layered Architecture.
4. MVC (Model-View-Controller) Architecture.
5. Event-Driven Architecture.
6. Microservices Architecture.
7. Serverless Architecture.

MVC Architecture

KIẾN TRÚC MVC

MVC Architecture

MVC Architecture Pattern



MVC Architecture

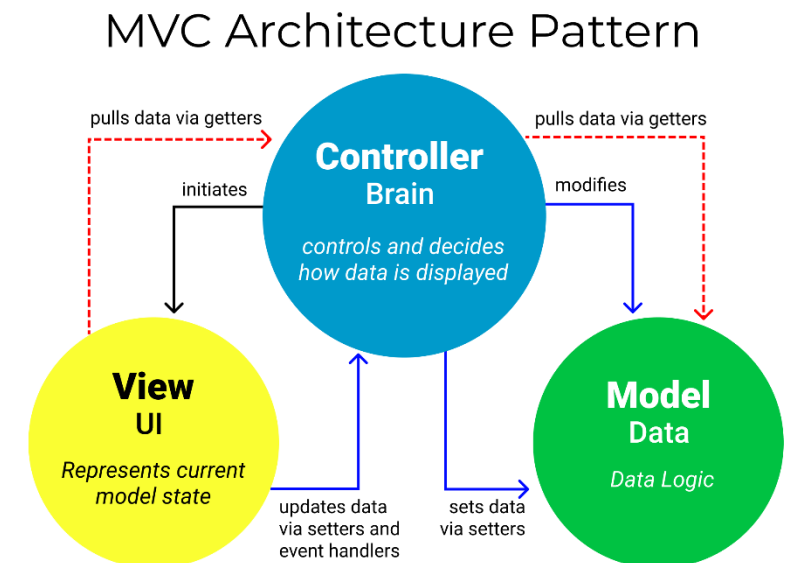
- MVC (Model-View-Controller) Architecture is a software design pattern that divides an application into three separate components:
 - + Model (M) – Manages data and business logic.
 - + View (V) – Handles the user interface and presentation.
 - + Controller (C) – Manages user interactions and acts as a bridge between Model and View.
- This separation improves code reusability, maintainability, and scalability by keeping data management, UI, and control logic independent.

MVC Architecture

- Kiến trúc MVC là một mẫu kiến trúc phần mềm giúp tổ chức ứng dụng thành ba thành phần riêng biệt:
 - + Thành phần Model (M) – Quản lý dữ liệu và logic nghiệp vụ.
 - + Thành phần View (V) – Hiển thị dữ liệu cho người dùng.
 - + Thành phần Controller (C) – Xử lý tương tác từ người dùng và điều phối dữ liệu giữa Model và View.
- MVC giúp tăng tính tái sử dụng, bảo trì dễ dàng và tách biệt rõ ràng giữa giao diện, xử lý nghiệp vụ và dữ liệu.

Components of MVC Architecture

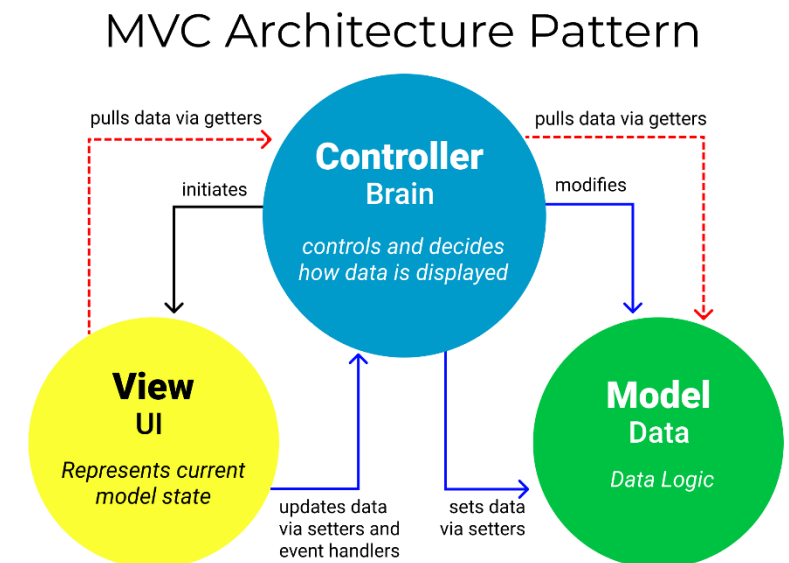
- Model (M) – Data Management
 - + Handles data storage, business rules, and processing logic.
 - + Does not concern itself with UI presentation.
 - + Example: In an e-commerce app, the Model stores product details, cart items, and user accounts.



Các thành phần trong MVC

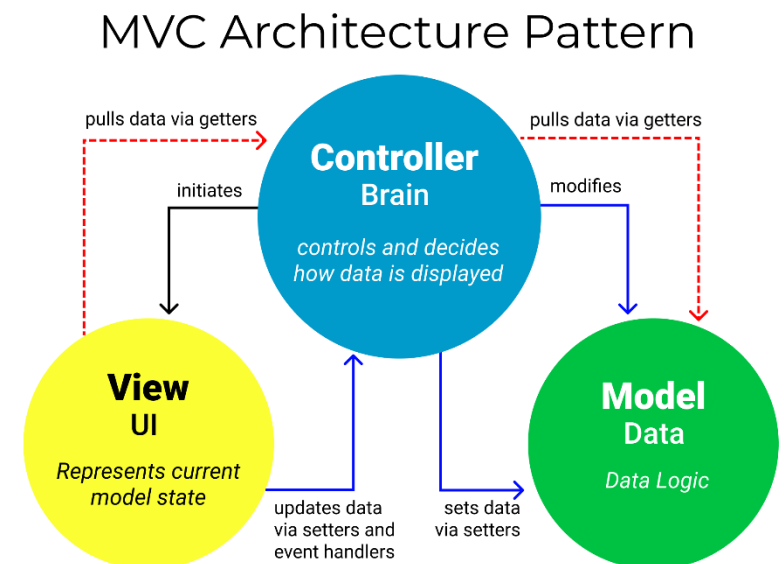
— Thành phần Model (M) – Quản lý dữ liệu

- + Chịu trách nhiệm lưu trữ, xử lý và quản lý dữ liệu của ứng dụng.
- + Không quan tâm đến cách dữ liệu được hiển thị trên giao diện.
- + Ví dụ: Trong ứng dụng thương mại điện tử, Model sẽ lưu trữ thông tin sản phẩm, giỏ hàng, người dùng.



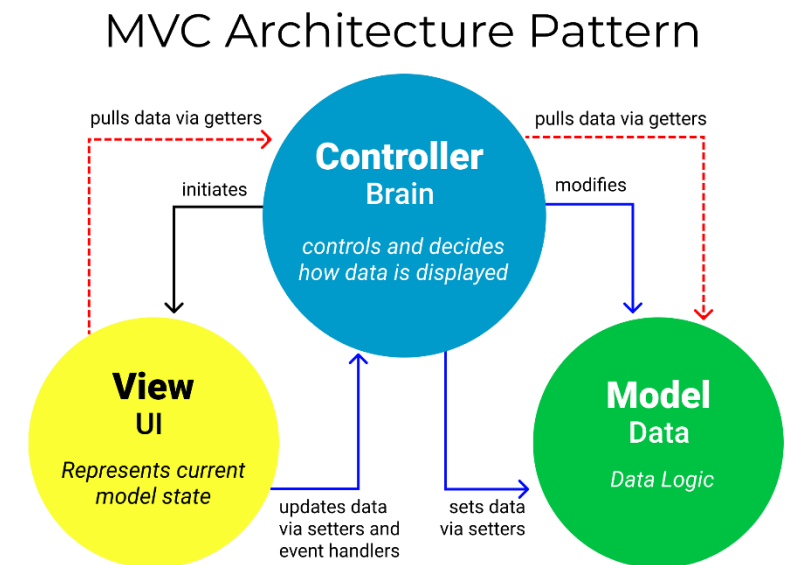
Components of MVC Architecture

- View (V) – User Interface
 - + Displays data from the Model to the user.
 - + Does not handle business logic; only responsible for UI rendering (HTML, CSS, UI components).
 - + Example: A webpage displaying a product list retrieved from the Model.



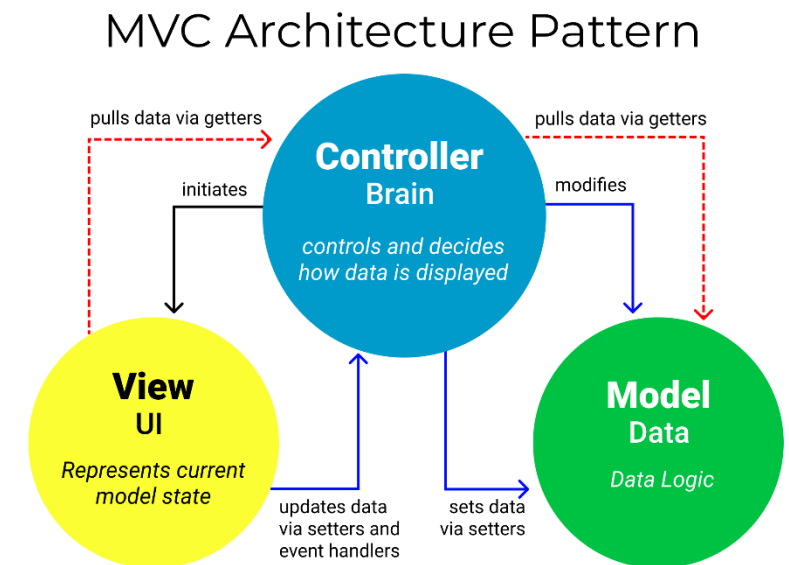
Các thành phần trong MVC

- Thành phần View (V) – Thành phần Giao diện người dùng
 - + Hiển thị dữ liệu từ Model ra màn hình cho người dùng.
 - + Không xử lý logic dữ liệu, chỉ nhận dữ liệu từ Model và hiển thị dưới dạng HTML, CSS, UI Components.
 - + Ví dụ: Một trang web hiển thị danh sách sản phẩm lấy từ Model.



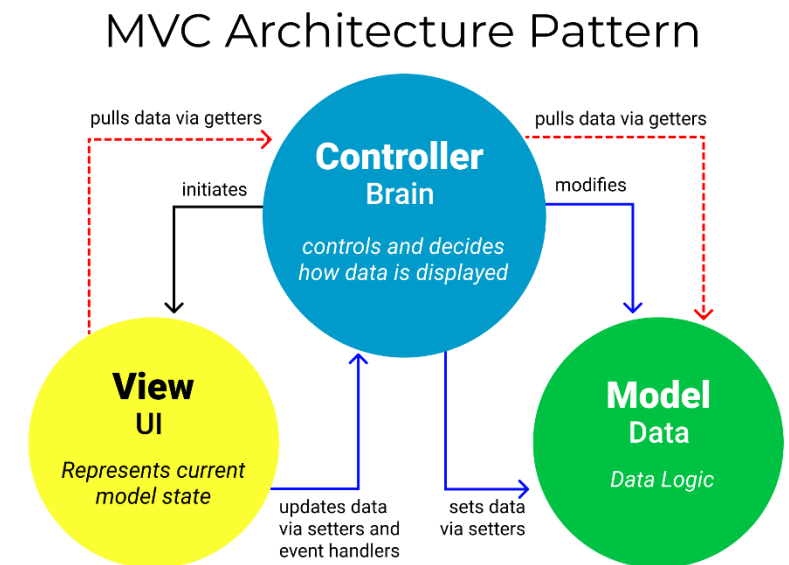
Components of MVC Architecture

- Controller (C) – Request Handling
 - + Takes user input (clicks, form submissions).
 - + Processes the request, updates the Model, and selects the appropriate View.
 - + Example: When a user clicks "Add to Cart," the Controller updates the cart and displays the updated view.



Các thành phần trong MVC

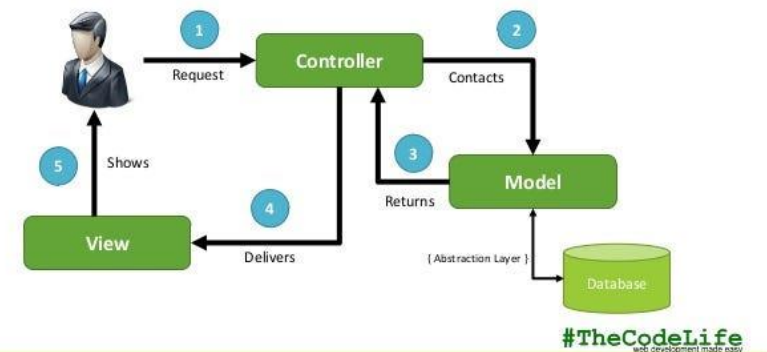
- Controller (C) – Điều khiển luồng xử lý
 - + Nhận yêu cầu từ người dùng (nhấp chuột, nhập dữ liệu).
 - + Xử lý yêu cầu, cập nhật Model và chọn View phù hợp để hiển thị kết quả.
 - + Ví dụ: Khi người dùng bấm "Mua hàng", Controller sẽ gọi Model để cập nhật giỏ hàng và hiển thị giao diện mới.



How MVC Works

1. User interacts with the View (UI).
2. View sends an event (user action) to the Controller.
3. Controller processes the event and updates the Model.
4. Model changes, and the updated data is sent to the View.
5. View displays the updated data to the user.

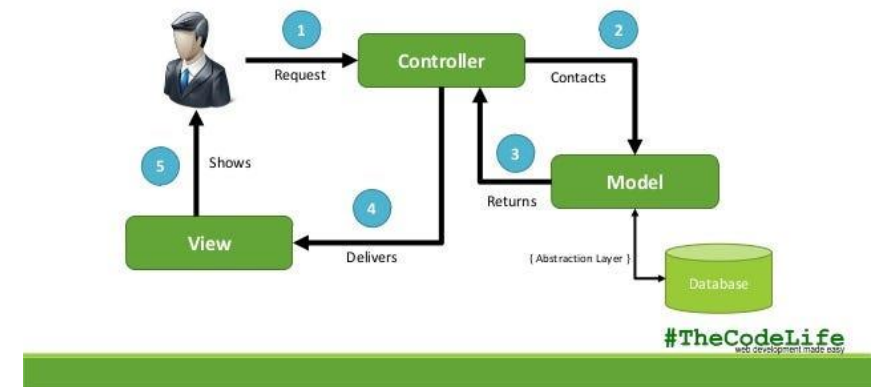
How it works



Cách hoạt động của MVC

1. Người dùng tương tác với thành phần giao diện (View).
2. View gửi sự kiện (event) đến thành phần Controller.
3. Thành phần Controller xử lý sự kiện, cập nhật thành phần Model nếu cần.
4. Thành phần Model thay đổi, gửi dữ liệu mới về thành phần View.
5. Thành phần View hiển thị dữ liệu cập nhật cho người dùng.

How it works



Advantages of MVC Architecture

1. Separation of Concerns – Clear separation between UI, business logic, and data.
2. High Code Reusability – The same Model can be used with multiple Views.
3. Better Testing & Maintainability – Easier to debug and modify each component independently.
4. Multi-platform Support – Can be adapted for Web, Mobile, and Desktop applications.

Ưu điểm của MVC Architecture

1. Tách biệt giữa giao diện, logic xử lý và dữ liệu → Dễ bảo trì, mở rộng.
2. Tái sử dụng code cao → Model có thể được sử dụng lại trong nhiều View khác nhau.
3. Tăng khả năng kiểm thử (Testing) → Có thể kiểm tra từng thành phần riêng biệt mà không ảnh hưởng đến thành phần khác.
4. Hỗ trợ nhiều nền tảng → Dễ dàng chuyển đổi giữa Web, Mobile và Desktop Apps.

Disadvantages of MVC Architecture

1. More Complex than Monolithic Architecture – Requires proper understanding and structure.
2. Synchronization Issues – Managing interactions between Model, View, and Controller can be challenging.
3. Not Ideal for Small Applications – Might introduce unnecessary complexity for simple projects.

Nhược điểm của MVC Architecture

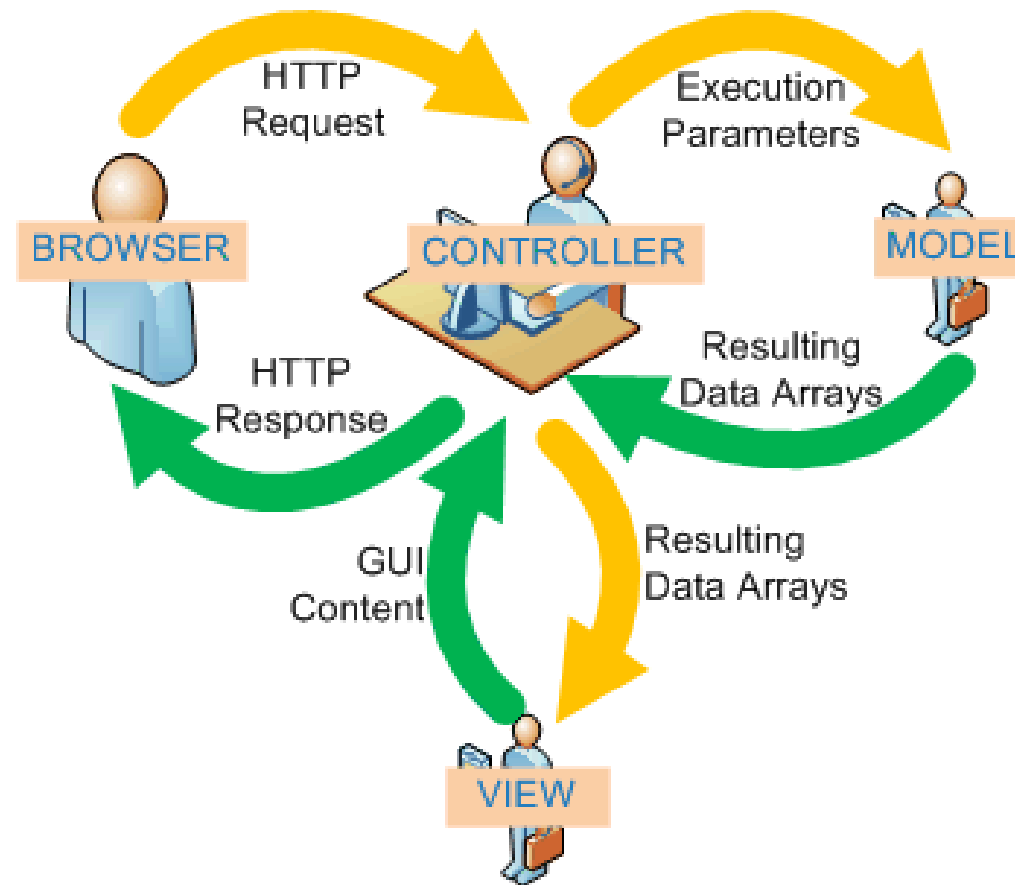
1. Phức tạp hơn Monolithic Architecture → Cần hiểu rõ cách tổ chức code đúng cách.
2. Khó đồng bộ giữa các phần (Model, View, Controller) → Nếu không quản lý tốt, dễ gây xung đột dữ liệu.
3. Không phù hợp với ứng dụng nhỏ → Nếu ứng dụng đơn giản, MVC có thể làm tăng độ phức tạp không cần thiết.

Real-World Examples of MVC

— Web Applications:

- + Django (Python), ASP.NET MVC (.NET), Spring MVC (Java), Ruby on Rails use MVC.
- + Example: E-commerce website
 - Model: Manages products, orders, and user data.
 - View: Displays product lists, order history.
 - Controller: Handles "Add to Cart" and checkout actions.

Real-World Examples of MVC



Ví dụ về MVC trong thực tế

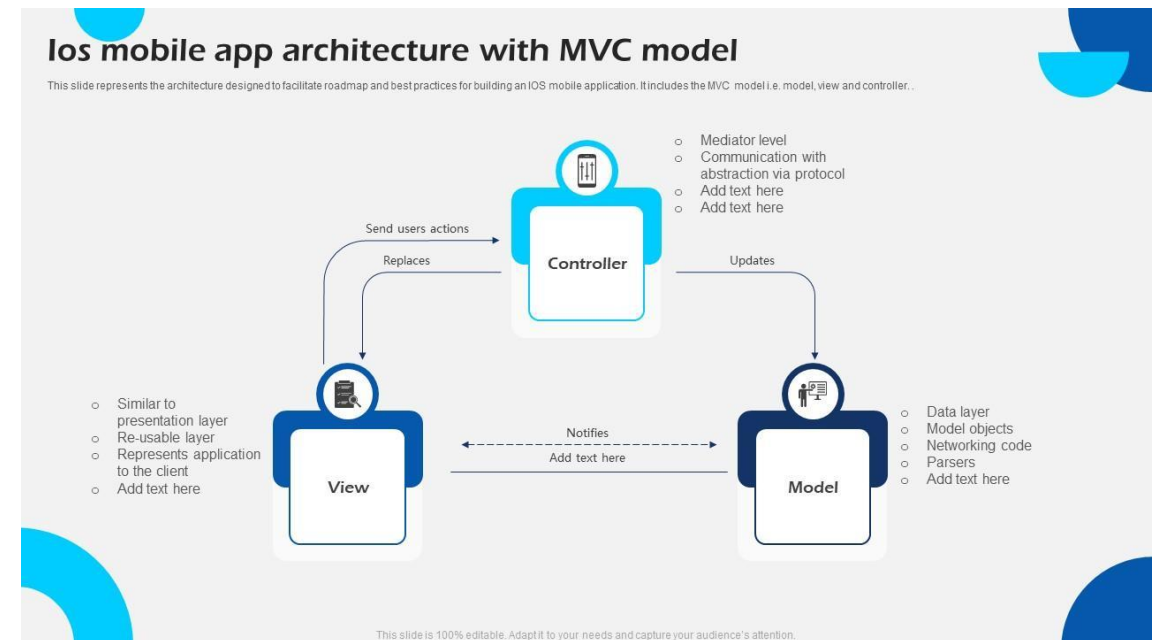
— Web Applications:

- + Django (Python), ASP.NET MVC (.NET), Spring MVC (Java), Ruby on Rails đều sử dụng MVC.
- + Ví dụ: Trang web thương mại điện tử
 - Model: Quản lý sản phẩm, đơn hàng, tài khoản người dùng.
 - View: Hiển thị danh sách sản phẩm, thông tin đơn hàng.
 - Controller: Xử lý khi người dùng nhấn "Thêm vào giỏ hàng".

Real-World Examples of MVC

— Mobile Applications:

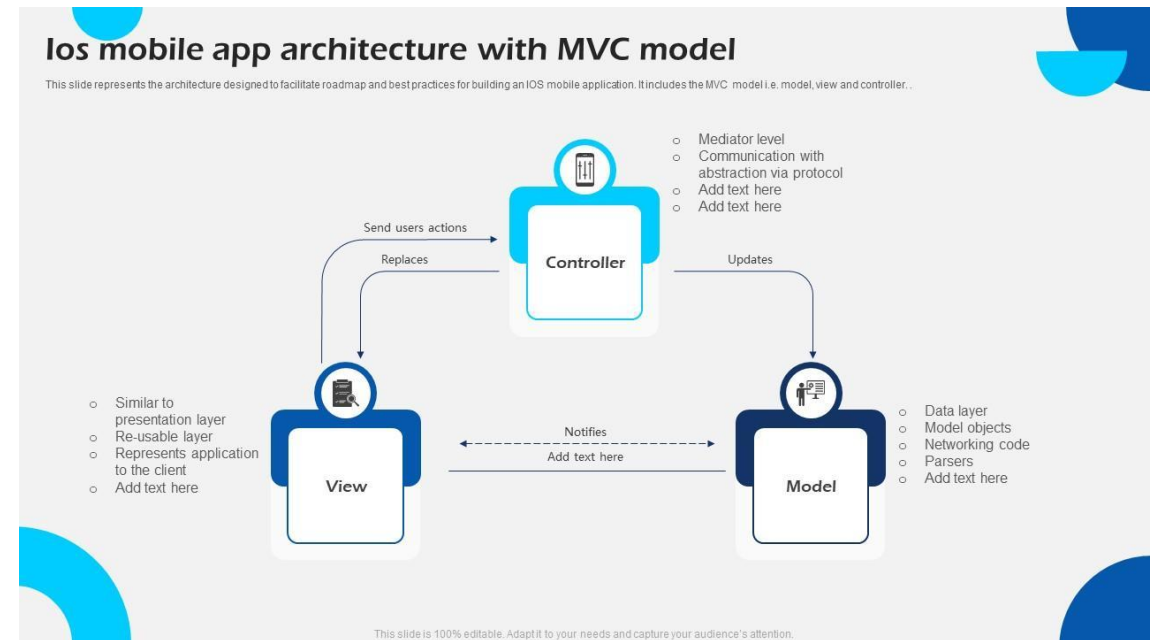
- + Android Apps use MVVM (a variant of MVC) to separate UI and business logic.
- + iOS Apps (Swift) use MVC in UIKit framework.



Ví dụ về MVC trong thực tế

— Mobile Applications:

- + Android Apps sử dụng MVVM (biến thể của MVC) để tách logic UI và dữ liệu.
- + iOS Apps (Swift) sử dụng MVC trong UIKit framework.



When to Use MVC?

- When separating UI and logic improves maintainability.
- Khi cần tách biệt giữa thành phần giao diện và thành phần logic xử lý.
- When developing medium to large-scale applications.
- Khi ứng dụng có quy mô trung bình hoặc lớn, cần bảo trì dễ dàng.
- When supporting multiple platforms (Web, Mobile, API).
- Khi cần hỗ trợ nhiều nền tảng (Web, Mobile, API).

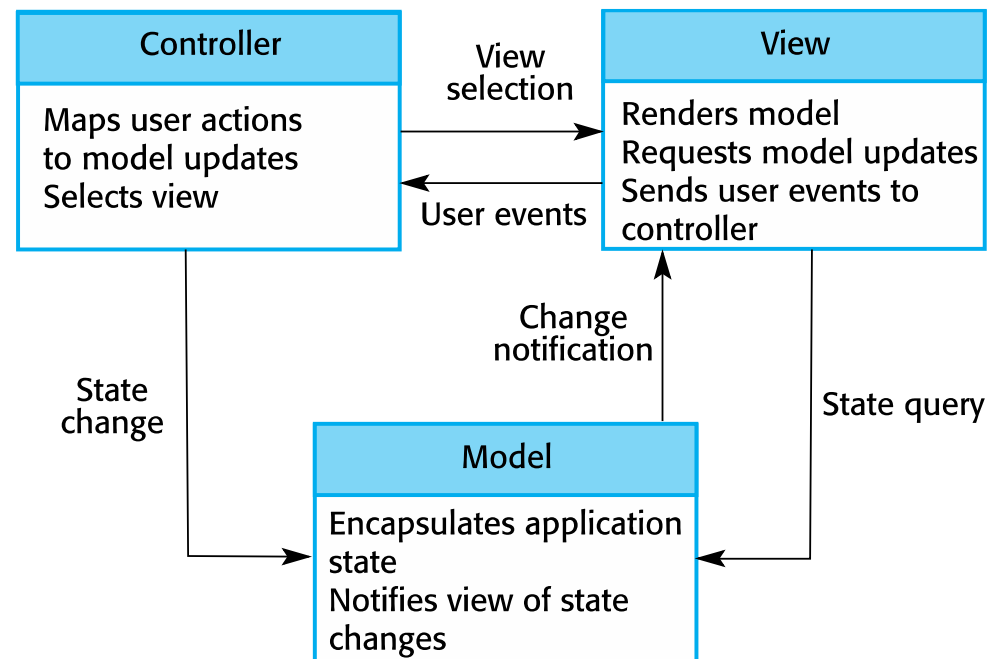
Conclusion – Kết luận

- MVC is a clear, maintainable, and scalable architecture widely used for web, mobile, and desktop applications.
- MVC là một mô hình kiến trúc rõ ràng, dễ bảo trì và linh hoạt, phù hợp cho các ứng dụng web, mobile và desktop.
- However, for small applications, it may introduce unnecessary complexity.
- Tuy nhiên, với những ứng dụng nhỏ, đôi khi MVC có thể làm tăng độ phức tạp không cần thiết.

MVC Architecture

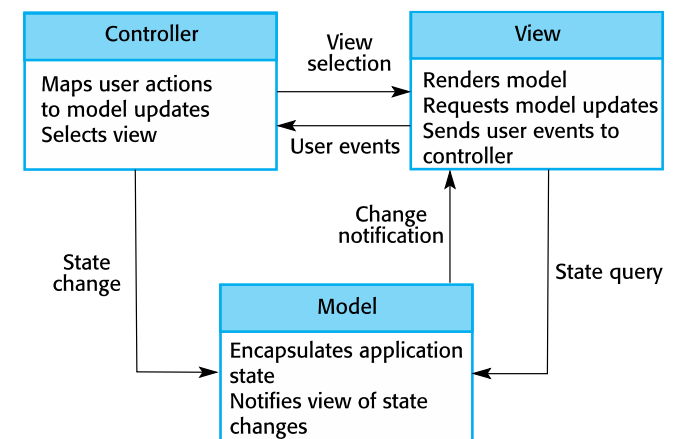
KIẾN TRÚC MVC

MVC Architecture – Kiến trúc MVC



MVC Architecture

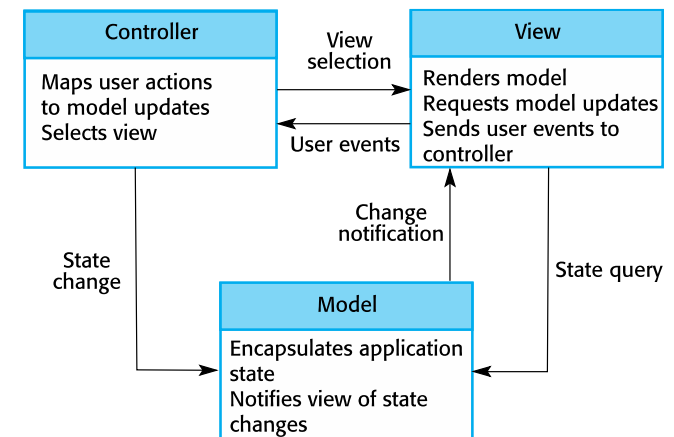
- The image represents the MVC (Model-View-Controller) architecture, a widely used design pattern in software development, especially for web and mobile applications.
- Hình này mô tả mô hình kiến trúc MVC (Model-View-Controller), một mẫu thiết kế phổ biến trong phát triển phần mềm, đặc biệt là phát triển ứng dụng web và mobile.



MVC Architecture

— Model (M) - Data Management

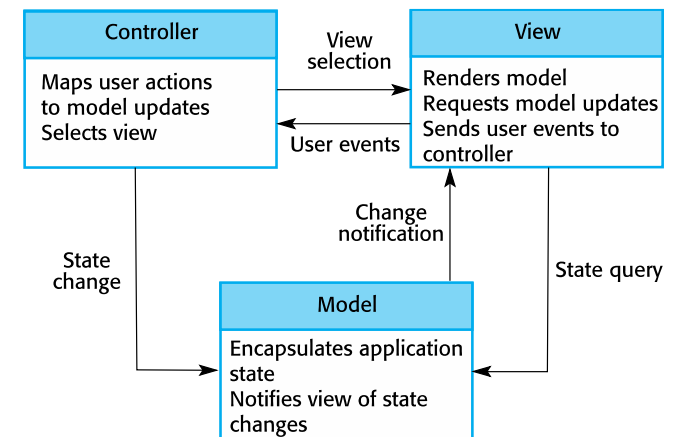
- + Responsible for storing and managing the application state.
- + Handles business logic and data processing.
- + Notifies the View when data changes so that the UI can be updated.



MVC Architecture

— Model (M) - Mô hình dữ liệu

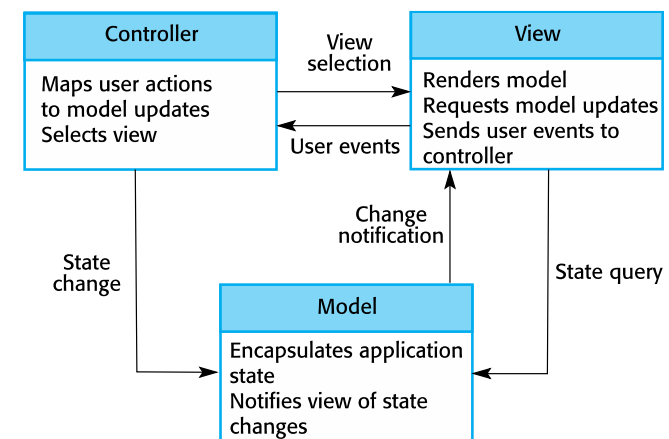
- + Chịu trách nhiệm lưu trữ, quản lý trạng thái của ứng dụng.
- + Thực hiện logic xử lý dữ liệu và cập nhật trạng thái.
- + Khi dữ liệu thay đổi, Model sẽ thông báo cho View biết để cập nhật giao diện.



MVC Architecture

— View (V) - User Interface

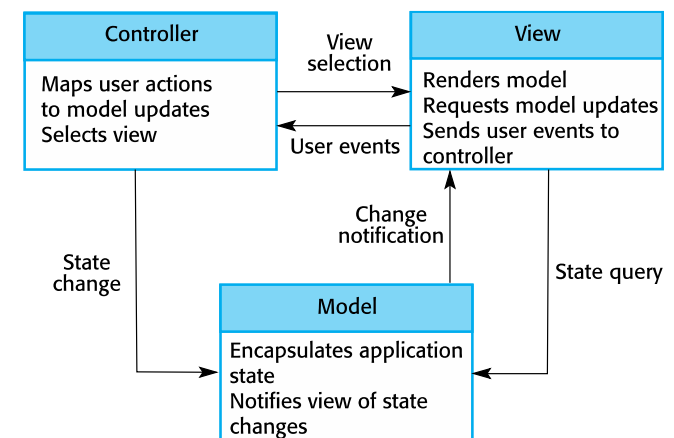
- + Renders the data from the Model for the user.
- + Requests updates from the Model when needed.
- + Sends user events (such as clicks or inputs) to the Controller.



MVC Architecture

— View (V) - Giao diện hiển thị

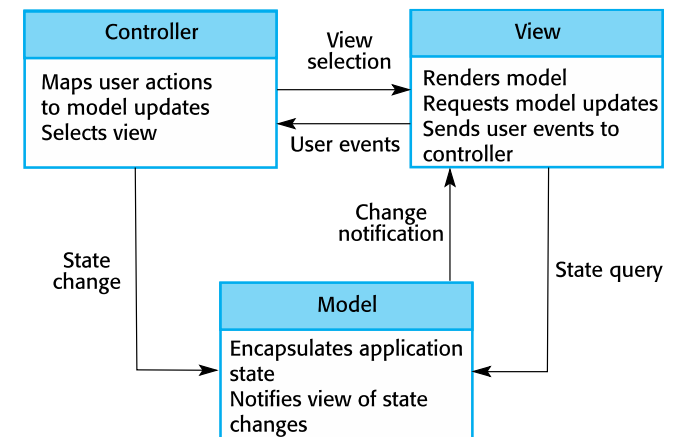
- + Hiển thị dữ liệu từ Model ra màn hình cho người dùng.
- + Nhận yêu cầu cập nhật dữ liệu từ Model.
- + Gửi sự kiện (event) của người dùng đến Controller.



MVC Architecture

— Controller (C) - Event Handler

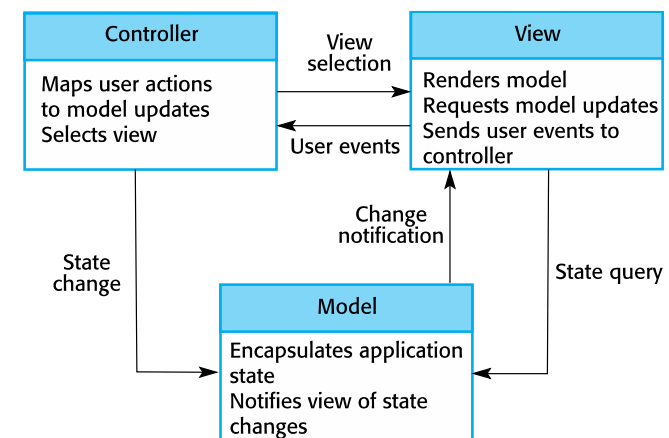
- + Receives user inputs and translates them into actions.
- + Updates the Model based on user interactions.
- + Chooses the appropriate View to display the updated data.



MVC Architecture

— Controller (C) - Bộ điều khiển

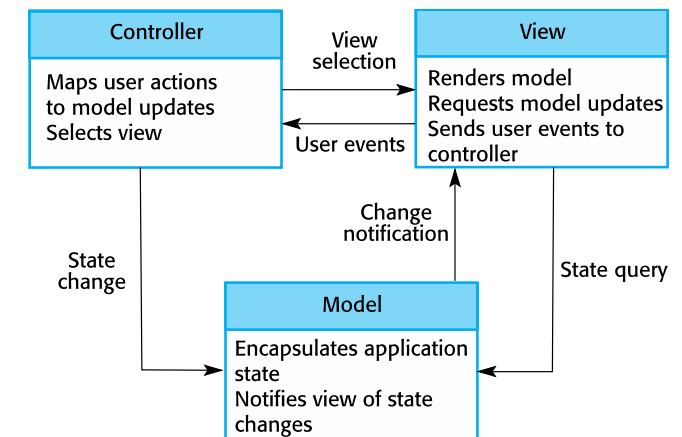
- + Nhận tương tác từ người dùng và chuyển đổi thành hành động.
- + Điều chỉnh dữ liệu trong Model dựa vào hành động của người dùng.
- + Chọn View phù hợp để hiển thị dữ liệu.



MVC Architecture

— How MVC Works:

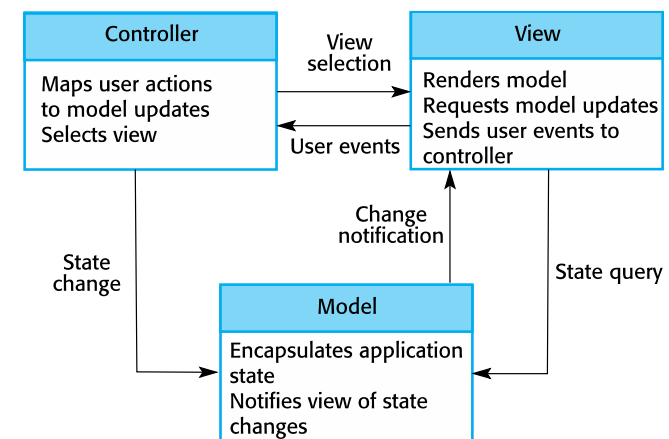
1. The user interacts with the View (e.g., clicks a button).
2. The View sends the event to the Controller.
3. The Controller processes the event and updates the Model.
4. The Model changes and notifies the View.
5. The View updates the UI to reflect the new data.



MVC Architecture

— Luồng hoạt động của MVC

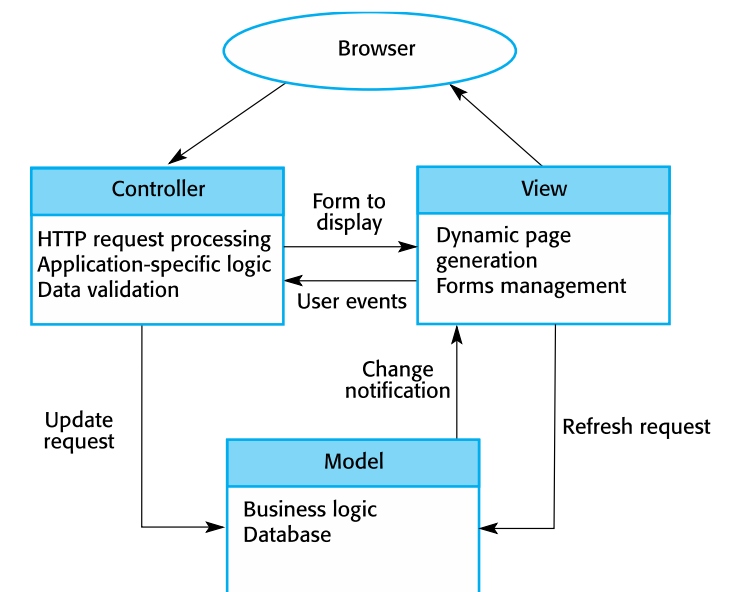
1. Người dùng thao tác trên giao diện (View).
2. View gửi sự kiện đến Controller.
3. Controller xử lý sự kiện và yêu cầu cập nhật dữ liệu từ Model.
4. Model cập nhật trạng thái ứng dụng và thông báo lại cho View.
5. View cập nhật lại giao diện để phản ánh dữ liệu mới từ Model.



MVC Architecture

— Example in a Web Application:

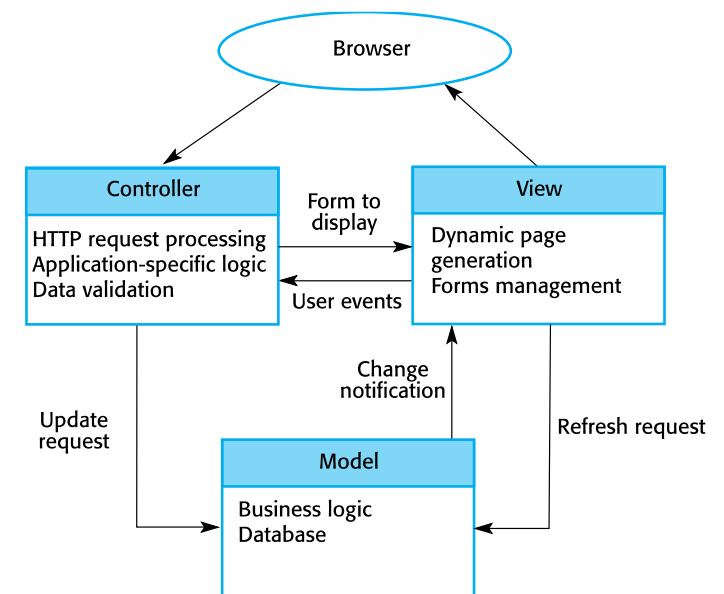
- + **Model:** Manages a shopping cart's data.
- + **View:** Displays the shopping cart to the user.
- + **Controller:** Handles actions like adding or removing items from the cart.



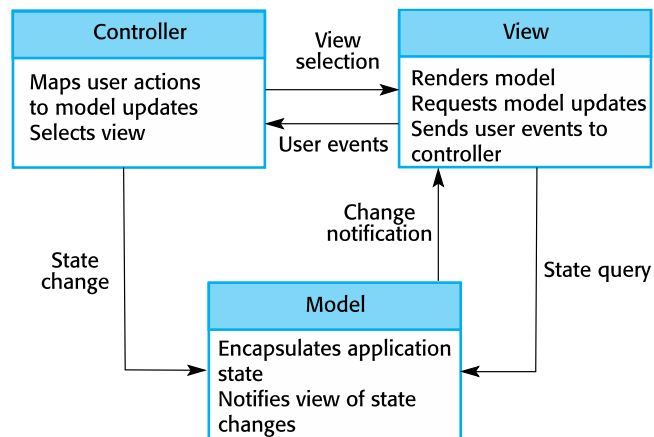
MVC Architecture

— Ví dụ trong ứng dụng web

- + Model: Quản lý danh sách sản phẩm trong giỏ hàng.
- + View: Giao diện hiển thị giỏ hàng cho người dùng.
- + Controller: Xử lý khi người dùng thêm hoặc xóa sản phẩm khỏi giỏ hàng.



MVC Architecture



- MVC helps structure code for better maintainability, scalability, and separation of concerns in application development.
- Mô hình MVC giúp tổ chức mã lệnh gọn gàng, dễ bảo trì và mở rộng ứng dụng.

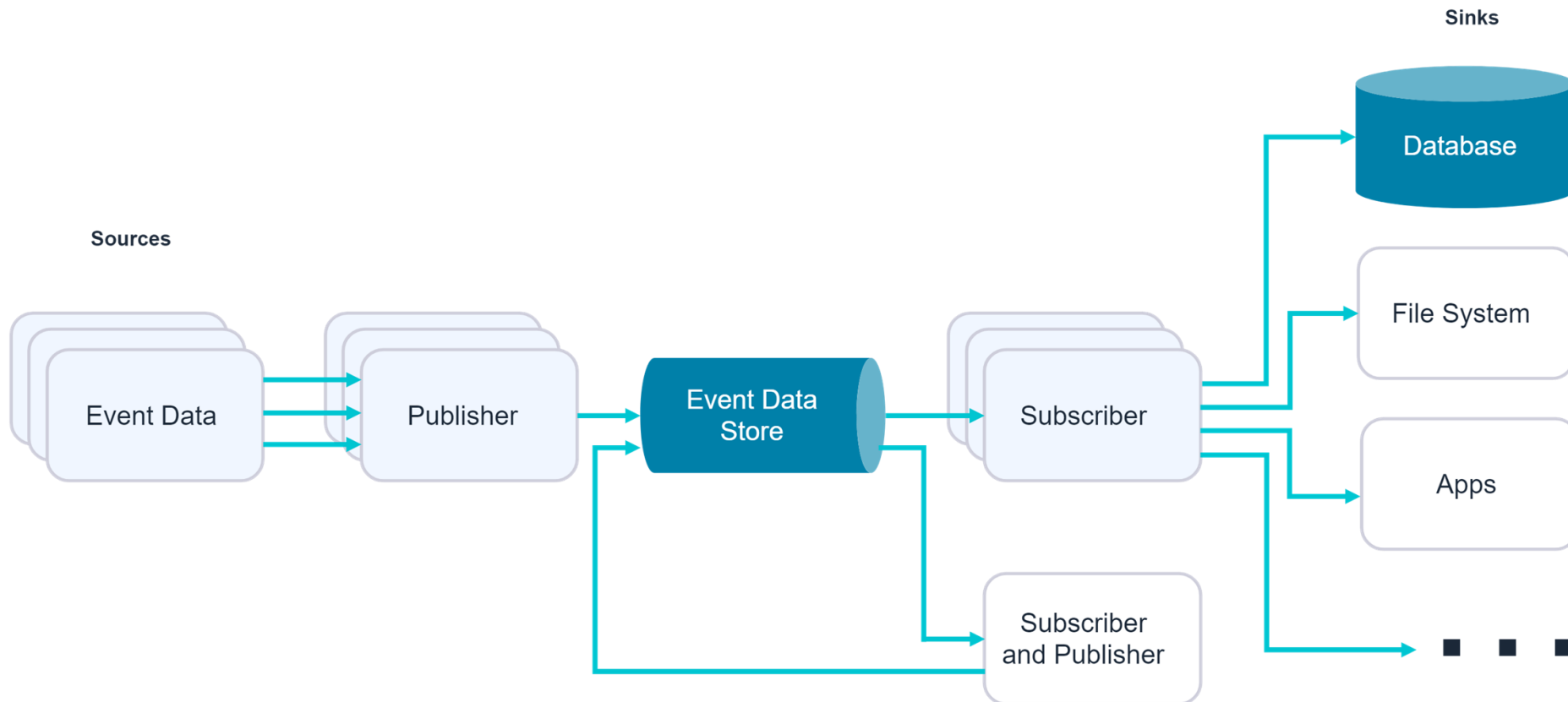
Common Software Architecture Patterns

1. Monolithic Architecture.
2. Client-Server Architecture.
3. Layered Architecture.
4. MVC (Model-View-Controller) Architecture.
5. Event-Driven Architecture.
6. Microservices Architecture.
7. Serverless Architecture.

Event-Driven Architecture

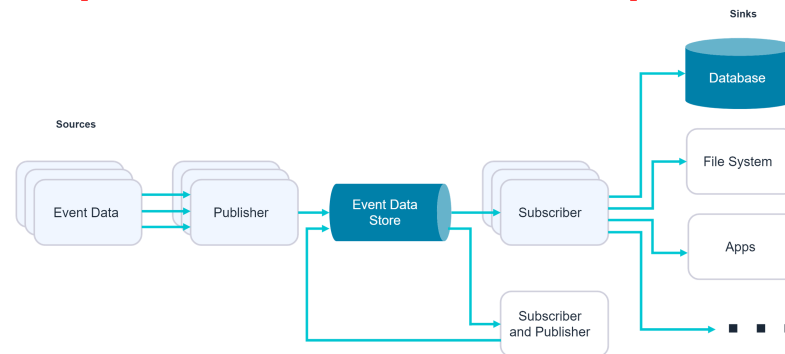
KIẾN TRÚC HƯỚNG SỰ KIỆN

Event-Driven Architecture



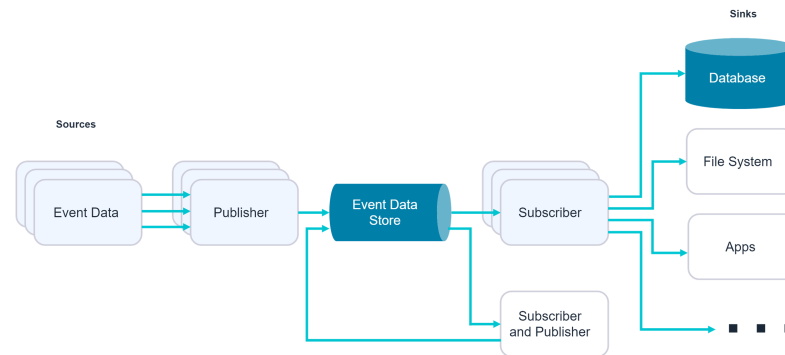
Event-Driven Architecture

- Event-Driven Architecture (EDA) is a software design pattern where system components communicate through events instead of direct calls. An event occurs when a change happens in the system, and different components react to it asynchronously.
- A component that generates an event is called an Event Producer, while components that respond to the event are called Event Consumers.



Kiến thức EDA

- Kiến trúc hướng sự kiện (EDA) là một mô hình kiến trúc phần mềm trong đó các thành phần trong hệ thống giao tiếp với nhau thông qua sự kiện (events) thay vì gọi trực tiếp.
- Một thành phần sẽ phát ra sự kiện (event producer) và các thành phần khác sẽ lắng nghe và phản hồi lại sự kiện đó (event consumers).



Components of EDA

— EDA consists of three main components:

1. **Event Producer**

- + Generates events when something changes in the system.
- + Example: When a user places an order, an "OrderPlaced" event is created.

2. **Event Broker (Message Broker)**

- + Acts as an intermediary to route events from Producers to Consumers.
- + Common brokers include Kafka, RabbitMQ, AWS SNS/SQS.

3. **Event Consumer**

- + Listens for events and reacts accordingly.
- + Example: When an "OrderPlaced" event is received, the system sends an order confirmation email.

Các thành phần of EDA

— Kiến trúc hướng sự kiện (EDA) bao gồm ba thành phần chính:

1. **Event Producer (Bộ tạo sự kiện)**

- + Tạo ra các sự kiện khi có một thay đổi xảy ra trong hệ thống.
- + Ví dụ: Khi người dùng đặt hàng, một sự kiện "OrderPlaced" sẽ được tạo ra.

2. **Event Broker (Bộ trung gian sự kiện)**

- + Làm nhiệm vụ phân phối sự kiện từ Producer đến các Consumer.
- + Có thể sử dụng Message Broker như Kafka, RabbitMQ, AWS SNS/SQS để truyền tải sự kiện.

3. **Event Consumer (Bộ tiêu thụ sự kiện)**

- + Nhận sự kiện và thực hiện hành động tương ứng.
- + Ví dụ: Khi nhận sự kiện "OrderPlaced", hệ thống gửi email xác nhận đơn hàng.

How Event-Driven Architecture Works

1. An event occurs (e.g., a customer places an order).
2. Producer emits an event (e.g., "OrderPlaced" is sent to the message broker).
3. Message Broker distributes the event to interested.
4. Consumers process the event (e.g., updating inventory, processing payment, sending an email).

Cách hoạt động của EDA

1. Một sự kiện xảy ra (ví dụ: khách hàng đặt hàng).
2. Producer phát sự kiện (ví dụ: "OrderPlaced" được gửi đến Message Broker).
3. Message Broker phân phối sự kiện đến các Consumer quan tâm.
4. Consumers xử lý sự kiện (ví dụ: gửi email, cập nhật tồn kho, tạo hóa đơn).

Advantages of EDA

1. High flexibility: Components are loosely coupled, making changes easier.
2. Scalability: New consumers can be added without modifying existing components.
3. Improved performance: Asynchronous processing reduces system load.
4. Real-time responsiveness: Enables systems to react instantly to events.

Ưu điểm của EDA

1. Tính linh hoạt cao: Các thành phần không bị ràng buộc chặt chẽ với nhau.
2. Khả năng mở rộng tốt: Dễ dàng thêm Consumer mới mà không ảnh hưởng hệ thống cũ.
3. Cải thiện hiệu suất: Giảm tải cho hệ thống nhờ xử lý bất đồng bộ (asynchronous).
4. Đáp ứng thời gian thực: Hệ thống có thể phản ứng ngay khi sự kiện xảy ra.

Disadvantages of EDA

1. Complex debugging: Difficult to track and trace event flows in large systems.
2. Dependency on Message Brokers: If the broker fails, events may be lost.
3. State management challenges: Keeping data consistent across multiple events requires careful design.

Nhược điểm của EDA

1. Khó kiểm soát luồng dữ liệu: Khi có quá nhiều sự kiện, việc debug và theo dõi rất phức tạp.
2. Phụ thuộc vào Message Broker: Nếu hệ thống trung gian bị lỗi, sự kiện có thể bị mất.
3. Quản lý trạng thái khó khăn: Cần cơ chế lưu trữ sự kiện để đảm bảo tính toàn vẹn dữ liệu.

Example of Event-Driven Architecture

- In an E-commerce application: When a customer places an order, the system generates an "OrderPlaced" event.
- Different Consumers react to the event:
 - + Inventory Service updates product stock.
 - + Payment Service processes the transaction.
 - + Email Service sends a confirmation email.
 - + Shipping Service initiates the delivery process.

Ví dụ về EDA

- Trong một ứng dụng thương mại điện tử (E-commerce): Khi khách hàng đặt hàng, hệ thống tạo sự kiện "OrderPlaced".
- Các Consumer phản ứng với sự kiện:
 - + Inventory Service giảm số lượng hàng tồn kho.
 - + Payment Service xử lý thanh toán.
 - + Email Service gửi xác nhận đơn hàng.
 - + Shipping Service bắt đầu quy trình giao hàng.

When to Use Event-Driven Architecture?

1. When handling multiple concurrent events asynchronously.
2. When scalability and flexibility are priorities.
3. When building real-time systems (e.g., stock trading, IoT, live notifications).
4. When integrating independent microservices efficiently.

Khi nào nên sử dụng EDA?

1. Khi hệ thống có nhiều sự kiện xảy ra đồng thời và cần xử lý bất đồng bộ.
2. Khi cần khả năng mở rộng cao, dễ dàng thêm tính năng mới.
3. Khi muốn hệ thống phản ứng theo thời gian thực (real-time).
4. Khi có nhiều thành phần độc lập nhưng vẫn cần giao tiếp với nhau.

Conclusion EDA

- Event-Driven Architecture is a powerful model that enables real-time, scalable, and loosely coupled systems.
- However, it requires careful event management and robust infrastructure to ensure reliability and consistency.

Tổng kết EDA

- Kiến trúc hướng sự kiện là một mô hình mạnh mẽ giúp hệ thống phản ứng nhanh, mở rộng linh hoạt và hoạt động bất đồng bộ.
- Tuy nhiên kiến trúc hướng sự kiện cũng đòi hỏi cơ chế quản lý sự kiện hiệu quả để đảm bảo dữ liệu chính xác và không bị mất mát.

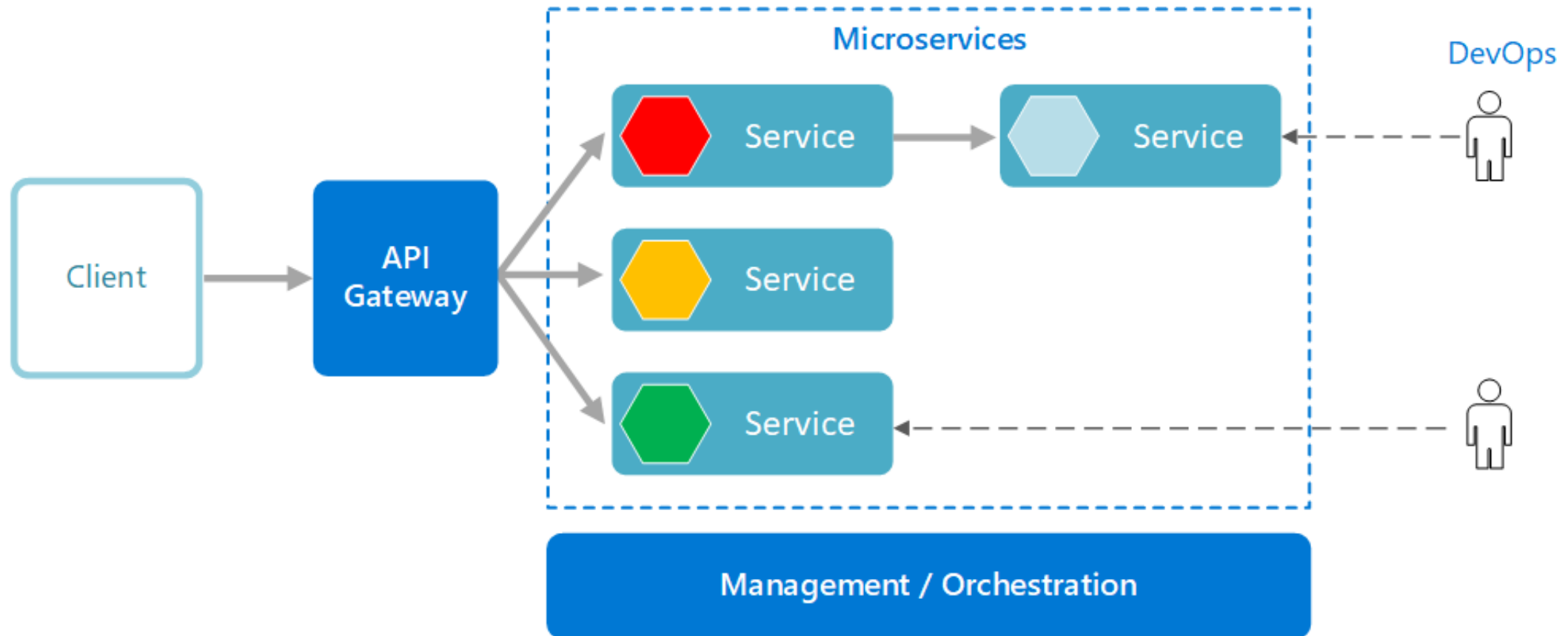
Common Software Architecture Patterns

1. Monolithic Architecture.
2. Client-Server Architecture.
3. Layered Architecture.
4. MVC (Model-View-Controller) Architecture.
5. Event-Driven Architecture.
6. **Microservices Architecture.**
7. Serverless Architecture.

Microservices Architecture

KIẾN TRÚC MICROSERVICES

Microservices Architecture



Microservices Architecture

- Microservices Architecture is a software design approach where an application is structured as a collection of small, independent services (microservices) that communicate with each other via APIs (typically HTTP, gRPC, or Message Queues).
- Each microservice is responsible for a specific functionality and can be developed, deployed, and scaled independently.

Kiến trúc Architecture

- Kiến trúc hướng dịch vụ là một phong cách thiết kế phần mềm trong đó hệ thống được chia thành nhiều dịch vụ nhỏ (microservices), hoạt động độc lập và giao tiếp với nhau thông qua giao thức API (thường là HTTP, gRPC, hoặc Message Queue).
- Mỗi dịch vụ (microservice) có một chức năng riêng biệt, có thể được phát triển, triển khai và mở rộng một cách độc lập.

Key Characteristics of MA

1. Service Independence: Each microservice handles a specific function, such as order processing, user authentication, or payment handling.
2. Independent Deployment: Microservices can be updated or scaled independently without affecting the entire system.
3. API Communication: Services interact through APIs using REST, gRPC, or Message Brokers (Kafka, RabbitMQ).

Đặc điểm của kiến trúc Microservices

1. Tách biệt dịch vụ: Mỗi dịch vụ (microservice) đảm nhận một nhiệm vụ cụ thể, ví dụ như xử lý đơn hàng, xác thực người dùng, thanh toán, v.v.
2. Triển khai độc lập: Các dịch vụ (microservices) có thể được phát triển, cập nhật hoặc mở rộng mà không ảnh hưởng đến toàn bộ hệ thống.
3. Giao tiếp qua API: Các dịch vụ (microservices) giao tiếp với nhau thông qua API (RESTful, gRPC, hoặc Message Broker như Kafka, RabbitMQ).

Key Characteristics of MA

4. Separate Databases: Each microservice can have its own database instead of sharing a central database.
5. Technology Diversity: Different microservices can use different programming languages and frameworks (e.g., Java, Node.js, Python).
6. High Scalability: The system can scale individual microservices instead of the entire application.

Đặc điểm của kiến trúc Microservices

4. Quản lý dữ liệu riêng biệt: Mỗi dịch vụ (microservice) có thể sử dụng cơ sở dữ liệu riêng (Database per service) thay vì chia sẻ một database lớn.
5. Công nghệ đa dạng: Các dịch vụ (microservices) có thể được xây dựng bằng nhiều ngôn ngữ lập trình và framework khác nhau (Java, Node.js, Python, v.v.).
6. Khả năng mở rộng cao: Dễ dàng mở rộng từng dịch vụ (microservice) thay vì phải mở rộng toàn bộ hệ thống.

Advantages of MA

1. High Flexibility: Each service can be developed and deployed independently.
2. Easier Maintenance & Updates: Fixing a bug in one microservice does not disrupt the whole system.
3. Scalability: Individual services can be scaled based on demand.

Ưu điểm của MA

1. Tính linh hoạt cao: Mỗi dịch vụ có thể được phát triển và triển khai riêng mà không ảnh hưởng đến các thành phần khác.
2. Dễ bảo trì và nâng cấp: Khi có lỗi, chỉ cần sửa một dịch vụ (microservice) mà không làm gián đoạn hệ thống.
3. Dễ mở rộng: Có thể mở rộng các thành phần riêng lẻ thay vì mở rộng toàn bộ hệ thống.

Advantages of MA

- 4. Technology Agnostic: Different microservices can use different tech stacks.
- 5. Improved Performance & Reliability: A failure in one microservice does not crash the entire application.

Ưu điểm của MA

4. Đa dạng công nghệ: Có thể sử dụng các công nghệ khác nhau cho từng dịch vụ (microservice).
5. Cải thiện hiệu suất và độ tin cậy: Nếu một dịch vụ (microservice) bị lỗi, nó không ảnh hưởng đến toàn bộ hệ thống.

Disadvantages of MA

1. Increased Complexity: Managing multiple microservices is harder than a single monolithic application.
2. Difficult Debugging & Monitoring: Requires strong logging and monitoring systems.
3. Communication Overhead: High dependency on well-designed APIs to avoid latency.
4. Higher Operational Costs: Requires advanced infrastructure, such as containers (Docker, Kubernetes) and API management tools.

Nhược điểm của MA

1. Tăng độ phức tạp: Việc quản lý nhiều dịch vụ cùng lúc đòi hỏi kỹ năng cao hơn so với kiến trúc nguyên khối (Monolithic).
2. Khó giám sát và debug: Phải có hệ thống logging và monitoring mạnh mẽ để theo dõi lỗi giữa nhiều microservices.
3. Giao tiếp giữa các dịch vụ: Phải thiết kế API hiệu quả để tránh độ trễ khi truyền dữ liệu giữa các microservices.
4. Chi phí vận hành cao: Yêu cầu hạ tầng mạnh hơn, bao gồm container (Docker, Kubernetes) và hệ thống quản lý API.

Ví dụ về Microservices Architecture

- Giả sử bạn có một hệ thống thương mại điện tử (E-commerce), bạn có thể chia thành các microservices như sau:
 - + User Service: Quản lý thông tin người dùng.
 - + Product Service: Quản lý sản phẩm.
 - + Order Service: Xử lý đơn hàng.
 - + Payment Service: Xử lý thanh toán.
 - + Notification Service: Gửi email hoặc tin nhắn thông báo cho khách hàng.
- Các microservices trên có thể chạy trên các server khác nhau, giao tiếp với nhau qua API hoặc Message Queue.

When to Use Microservices?

1. When building large, scalable systems.
2. When frequent updates and deployments are required.
3. When supporting multiple technologies and frameworks.
4. When working with large development teams that need to work on different parts of the system separately.

Khi nào nên sử dụng Microservices?

1. Hệ thống lớn, cần mở rộng nhanh chóng.
2. Dự án yêu cầu linh hoạt và cập nhật liên tục.
3. Cần hỗ trợ nhiều công nghệ khác nhau.
4. Có đội ngũ phát triển lớn, cần làm việc trên các phần khác nhau của hệ thống.

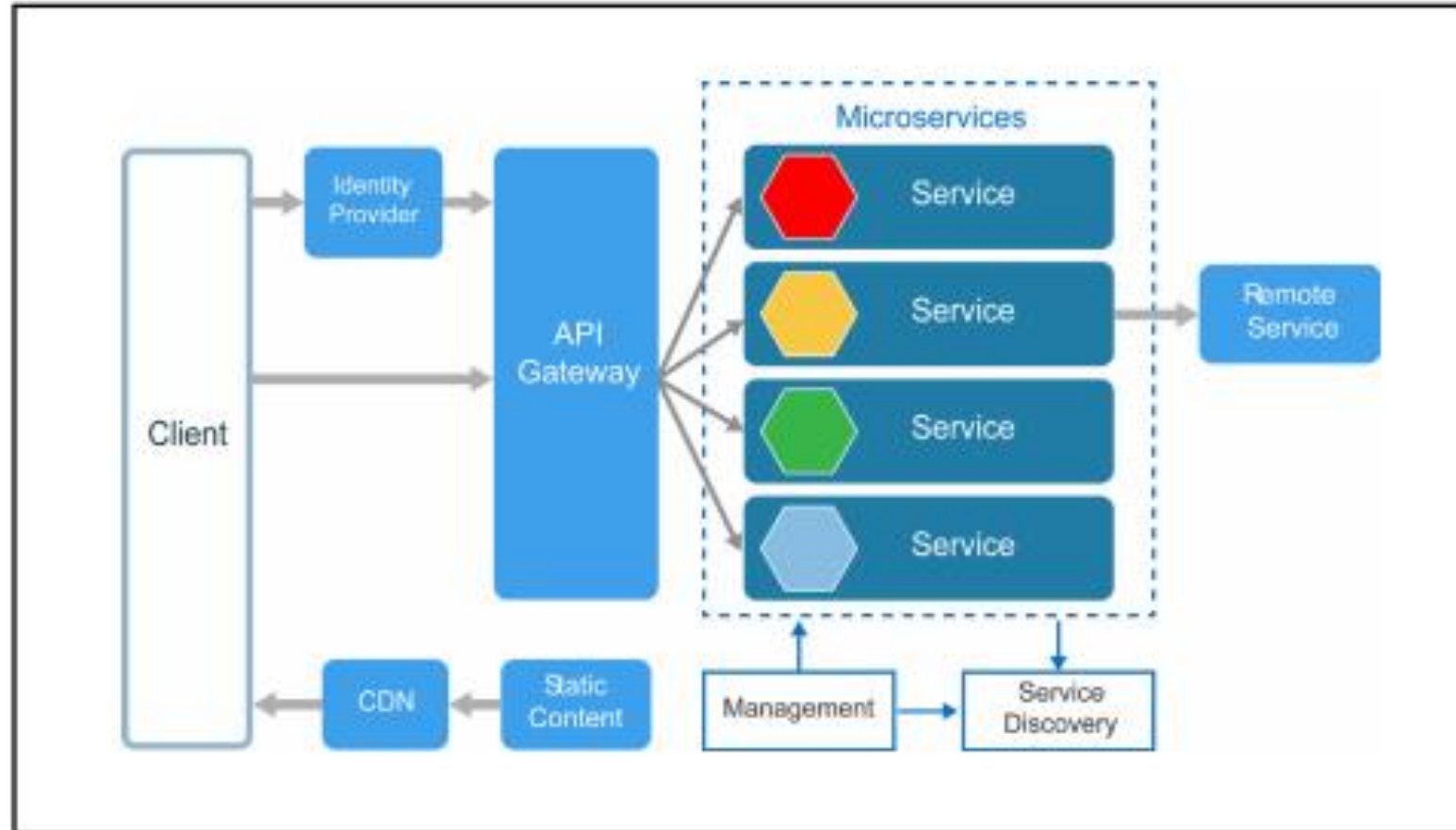
Conclusion of MA

- Microservices Architecture is a powerful approach for building scalable, flexible, and maintainable systems.
- However, it also introduces complexity in deployment, monitoring, and communication between services. Proper tooling and management strategies are necessary to handle these challenges effectively.

Tổng kết về MA

- Microservices Architecture là một giải pháp mạnh mẽ cho các hệ thống lớn, giúp tăng khả năng mở rộng, bảo trì và hiệu suất.
- Tuy nhiên, Microservices Architecture cũng đòi hỏi kỹ năng quản lý, triển khai phức tạp và công cụ hỗ trợ phù hợp.

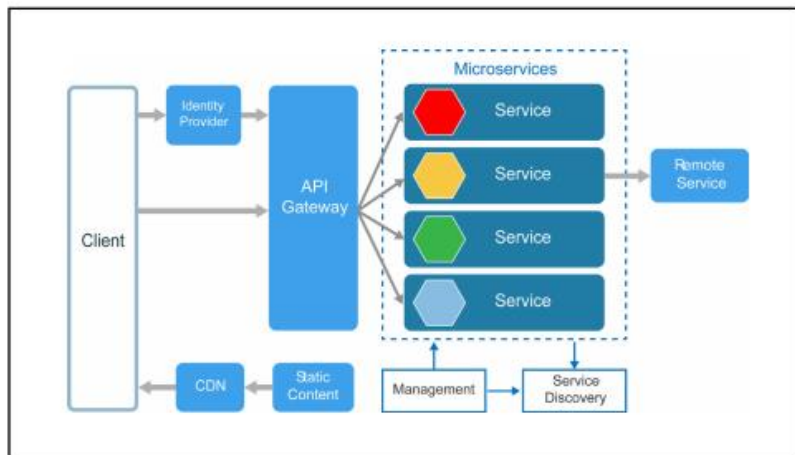
Microservices Architecture Example



Microservices Architecture Example

— Các thành phần chính trong sơ đồ

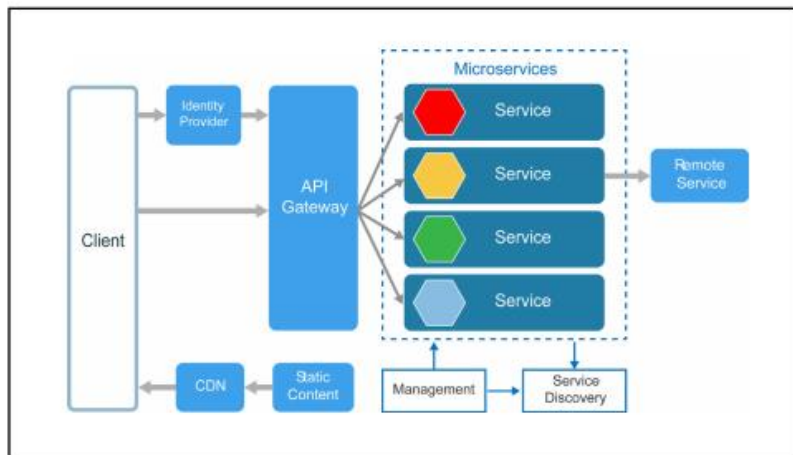
+ **Client** (Người dùng hoặc ứng dụng phía khách)



- Đây là ứng dụng hoặc thiết bị của người dùng (trình duyệt, ứng dụng di động, API của hệ thống khác).
- Gửi yêu cầu đến hệ thống thông qua API Gateway.

Microservices Architecture Example

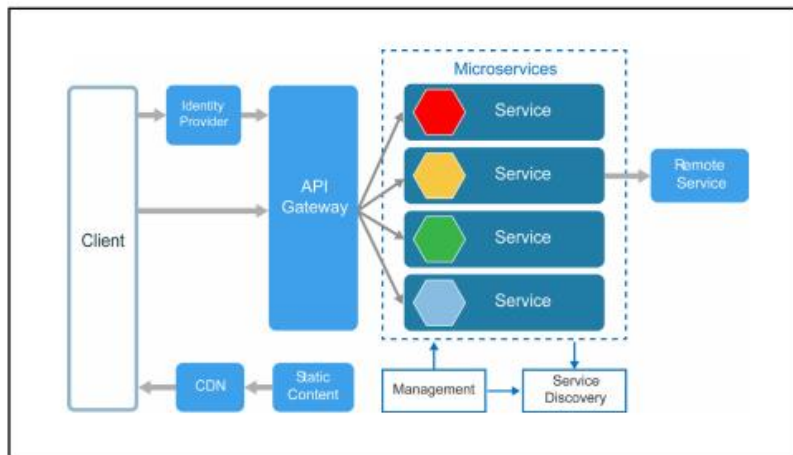
— Các thành phần chính trong sơ đồ
+ **API Gateway**



- Đóng vai trò trung gian giữa Client và các Microservices.
- **Chịu trách nhiệm:**
 - Chuyển tiếp yêu cầu đến đúng microservice.
 - **Xác thực thông qua Identity Provider.**
 - Kết hợp dữ liệu từ nhiều microservices nếu cần.
 - **Cân bằng tải (Load Balancing).**

Microservices Architecture Example

- Các thành phần chính trong sơ đồ
- + Identity Provider

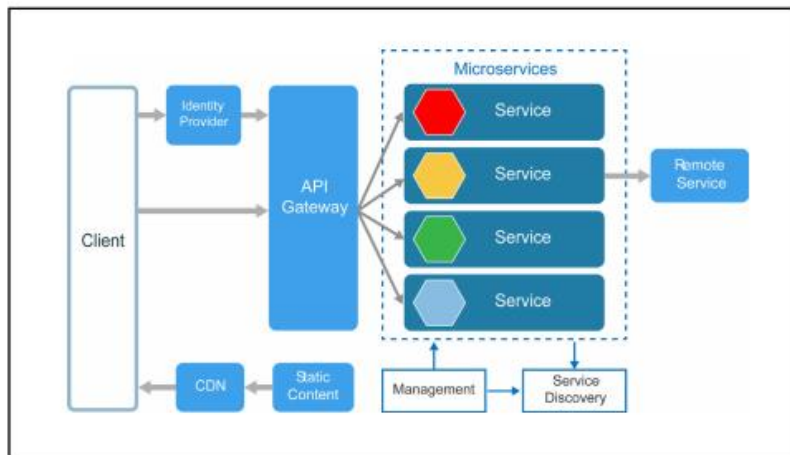


- Quản lý xác thực và ủy quyền người dùng.
- Đảm bảo rằng chỉ những người dùng hợp lệ mới có thể truy cập vào hệ thống.

Microservices Architecture Example

— Các thành phần chính trong sơ đồ

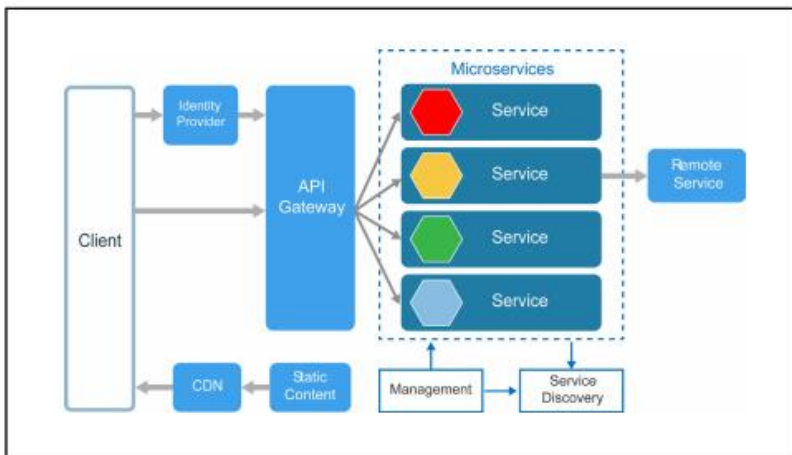
+ **CDN (Content Delivery Network) & Static Content**



- **CDN:** Lưu trữ và phân phối nội dung tĩnh như hình ảnh, CSS, JavaScript để tăng tốc tải trang.
- **Static Content:** Chứa các tệp tĩnh của hệ thống.

Microservices Architecture Example

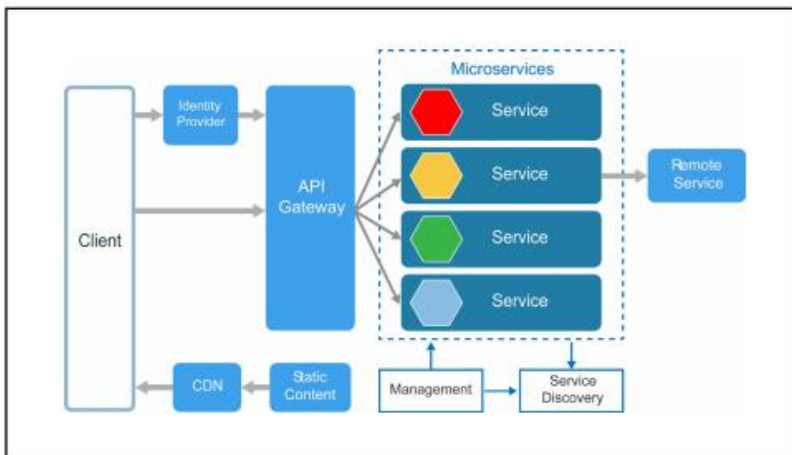
- Các thành phần chính trong sơ đồ
- + **Microservices**



- Hệ thống được chia thành nhiều dịch vụ nhỏ (Microservices), mỗi dịch vụ đảm nhận một nhiệm vụ riêng.
- Các microservices trong sơ đồ có màu sắc khác nhau, biểu thị tính độc lập.
- Chúng giao tiếp với nhau qua API.

Microservices Architecture Example

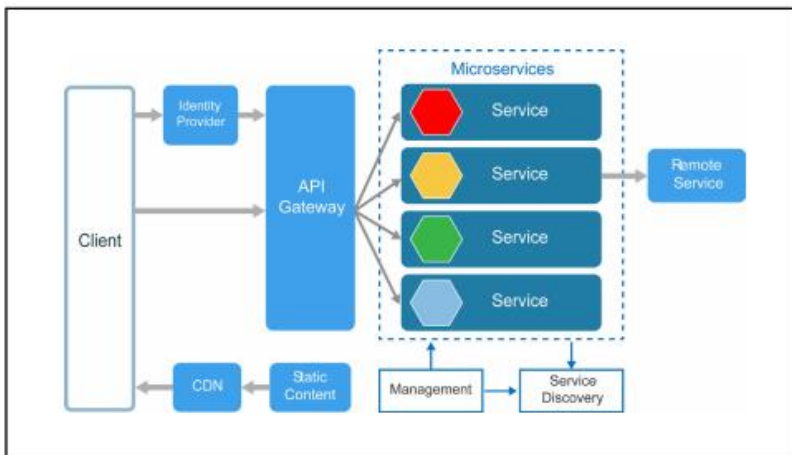
- Các thành phần chính trong sơ đồ
- + **Remote Service**



- Một số microservices có thể kết nối với các dịch vụ bên ngoài (ví dụ: cổng thanh toán, hệ thống phân tích dữ liệu).

Microservices Architecture Example

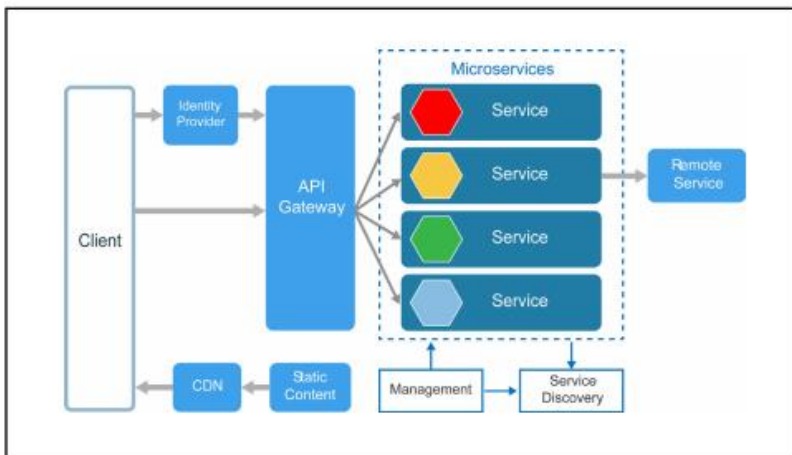
- Các thành phần chính trong sơ đồ
- + **Service Discovery**



- Cho phép các microservices tìm thấy nhau mà không cần biết địa chỉ cố định.
- Hỗ trợ tự động cân bằng tải và mở rộng.

Microservices Architecture Example

— Các thành phần chính trong sơ đồ
+ **Management**

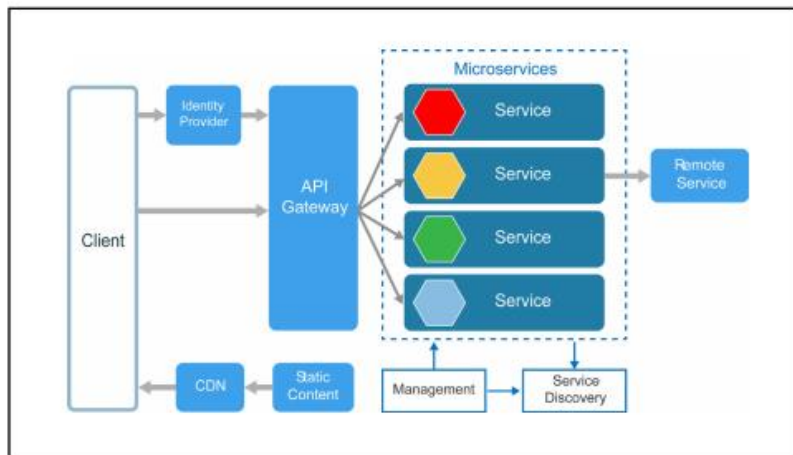


- Cung cấp các công cụ để theo dõi, kiểm soát và quản lý các microservices.
- Bao gồm logging, monitoring, scaling và debugging.

Microservices Architecture Example

— Luồng hoạt động trong hệ thống

1. Client gửi yêu cầu đến API Gateway.

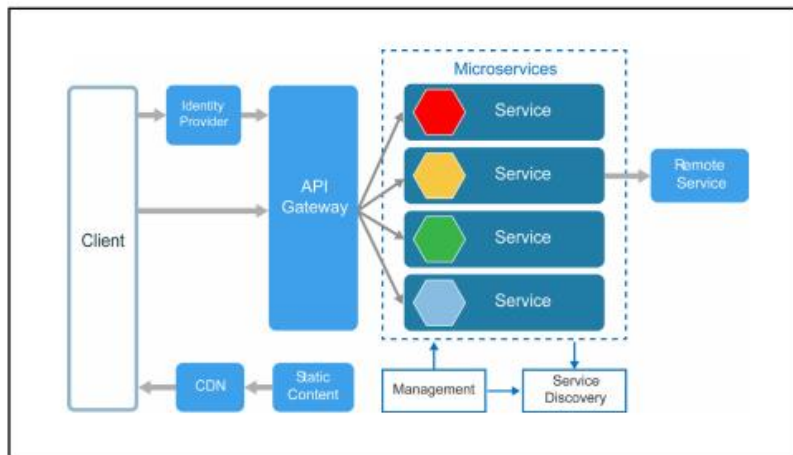


2. API Gateway:

- Xác thực yêu cầu với Identity Provider.
- Kiểm tra dữ liệu tĩnh trên CDN hoặc Static Content nếu cần.
- Chuyển tiếp yêu cầu đến microservice phù hợp.

Microservices Architecture Example

— Luồng hoạt động trong hệ thống



3. Microservices thực hiện xử lý logic và có thể gọi các microservices khác hoặc Remote Service.
4. Service Discovery hỗ trợ tìm kiếm và quản lý các microservices.
5. Management theo dõi hoạt động hệ thống để đảm bảo hiệu suất.

Microservices Architecture Example

— Ý nghĩa của sơ đồ

- + **API Gateway** đóng vai trò trung gian quan trọng, giúp hệ thống linh hoạt hơn.
 - + **Microservices** hoạt động độc lập, có thể triển khai, cập nhật mà không ảnh hưởng đến toàn hệ thống.
 - + **CDN** giúp cải thiện hiệu suất bằng cách phân phối nội dung tĩnh nhanh hơn.
 - + **Service Discovery** giúp hệ thống tự động mở rộng và quản lý tốt hơn.
- Sơ đồ này thể hiện một hệ thống Microservices hiện đại, có khả năng mở rộng cao và tối ưu hiệu suất.

Common Software Architecture Patterns

1. Monolithic Architecture.
2. Client-Server Architecture.
3. Layered Architecture.
4. MVC (Model-View-Controller) Architecture.
5. Event-Driven Architecture.
6. Microservices Architecture.
7. Serverless Architecture.

Serverless Architecture

KIẾN TRÚC KHÔNG MÁY CHỦ

Serverless Architecture

- Serverless Architecture is a software design pattern where developers do not need to manage servers. Instead of deploying and maintaining infrastructure, applications run on cloud provider-managed infrastructure (AWS, Google Cloud, Azure, etc.), and resources scale automatically based on demand.
- Despite the name "serverless," servers are still used, but they are fully managed by cloud providers, allowing developers to focus on coding instead of infrastructure management.

Serverless Architecture

- Serverless Architecture (Kiến trúc không máy chủ) là một mô hình kiến trúc trong đó nhà phát triển không cần quản lý máy chủ. Thay vì triển khai và duy trì server, ứng dụng sẽ chạy trên hạ tầng cloud của nhà cung cấp dịch vụ (AWS, Google Cloud, Azure, v.v.) và tài nguyên sẽ tự động mở rộng theo nhu cầu.
- Mặc dù có tên "serverless", thực tế hệ thống vẫn sử dụng server, nhưng việc quản lý server hoàn toàn do nhà cung cấp cloud xử lý, giúp lập trình viên tập trung vào phát triển ứng dụng thay vì quản lý cơ sở hạ tầng.

Key Components of SA

- FaaS (Function as a Service)
 - + Allows running individual functions without managing servers.
 - + Examples: AWS Lambda, Google Cloud Functions, Azure Functions.
- BaaS (Backend as a Service)
 - + Provides pre-built backend services like Authentication, Database, Storage, API Gateway.
 - + Examples: Firebase, AWS Cognito, Supabase.

Thành phần chính của SA

— FaaS (Function as a Service)

- + Cho phép chạy các hàm (functions) mà không cần quản lý server.
- + Ví dụ: AWS Lambda, Google Cloud Functions, Azure Functions.

— BaaS (Backend as a Service)

- + Cung cấp các dịch vụ backend có sẵn như Authentication, Database, Storage, API Gateway.
- + Ví dụ: Firebase, AWS Cognito, Supabase.

How Serverless Architecture Works

1. User sends a request to the API Gateway.
2. API Gateway triggers a serverless function (FaaS) to process the request.
3. The function processes data and interacts with backend services (BaaS).
4. The result is returned to the user, and resources are automatically released.

Cách hoạt động của SA

1. Người dùng gửi yêu cầu đến API Gateway.
2. API Gateway kích hoạt một hàm serverless (FaaS) để xử lý yêu cầu.
3. Hàm xử lý dữ liệu và có thể tương tác với cơ sở dữ liệu (BaaS).
4. Kết quả được gửi lại cho người dùng và tài nguyên được tự động giải phóng sau khi thực thi xong.

Advantages of Serverless Architecture

1. No server management required → Reduces infrastructure maintenance costs.
2. Automatic scaling (Auto-scaling) → The system can handle millions of requests without manual configuration.
3. Pay-as-you-go pricing → You only pay for the execution time of functions.
4. Fast deployment → Developers can focus on writing code rather than configuring infrastructure.

Ưu điểm của SA

1. Không cần quản lý server → Giảm chi phí bảo trì, vận hành.
2. Tự động mở rộng (Auto-scaling) → Hệ thống có thể xử lý hàng triệu yêu cầu mà không cần cấu hình thủ công.
3. Trả tiền theo lượng sử dụng → Không tốn phí nếu không có yêu cầu nào chạy.
4. Triển khai nhanh chóng → Dễ dàng phát triển các tính năng mà không cần lo về hạ tầng.

Disadvantages of SA

1. Inconsistent performance → Cold starts may cause latency if functions are inactive.
2. Difficult debugging & monitoring → Since everything runs in the cloud, debugging is more challenging.
3. Execution time limits → FaaS functions often have execution time constraints (e.g., AWS Lambda has a 15-minute limit).
4. Cloud vendor lock-in → Moving from one cloud provider to another can be complex.

Nhược điểm của SA

1. Hiệu suất không ổn định → Có thể bị độ trễ (cold start) khi hàm không được sử dụng thường xuyên.
2. Khó kiểm soát và debug → Vì chạy trên cloud, khó theo dõi lỗi hơn so với hệ thống truyền thống.
3. Giới hạn thời gian thực thi → FaaS thường có giới hạn thời gian chạy (ví dụ: AWS Lambda tối đa 15 phút).
4. Phụ thuộc vào nhà cung cấp cloud → Việc di chuyển hệ thống từ một nhà cung cấp cloud sang nhà cung cấp khác có thể phức tạp.

Real-World Applications of SA

- Web & API applications → Building RESTful APIs without managing servers.
- Real-time data processing → Streaming data from IoT devices, logs, or analytics.
- Chatbots & virtual assistants → Serverless functions can process messages efficiently.
- AI/ML applications → Image processing, speech recognition, and NLP tasks without dedicated infrastructure

Ứng dụng thực tế của SA

- Ứng dụng web và API → Xây dựng các API RESTful mà không cần server cố định.
- Xử lý dữ liệu theo thời gian thực → Streaming dữ liệu từ IoT, logs, phân tích dữ liệu lớn.
- Chatbot và trợ lý ảo → Serverless giúp xử lý tin nhắn, phản hồi người dùng một cách nhanh chóng.
- Ứng dụng AI/ML → Xử lý hình ảnh, nhận diện giọng nói mà không cần hạ tầng máy chủ phức tạp.

When to Use Serverless Architecture?

- When you want to reduce infrastructure costs and avoid server management.
- When you need a highly scalable system that adapts to traffic fluctuations.
- When building applications with irregular traffic (e.g., live events, flash sales).
- When processing real-time data streams without maintaining complex backend services.

Khi nào nên sử dụng SA?

- Khi muốn giảm chi phí hạ tầng và không cần quản lý server.
- Khi cần ứng dụng có thể mở rộng linh hoạt mà không tốn công sức quản lý tài nguyên.
- Khi cần ứng dụng có lưu lượng truy cập thay đổi thất thường (ví dụ: sự kiện trực tuyến, flash sales).
- Khi cần xử lý dữ liệu theo thời gian thực mà không cần hệ thống phức tạp.

Conclusion

- Serverless Architecture is a modern, flexible, and cost-efficient solution that enables applications to scale automatically without managing servers.
- However, it may not be ideal for applications requiring stable performance or long-running processes.
- Serverless is becoming a mainstream approach for cloud-native applications and scalable solutions.

Kết luận

- Serverless Architecture là một giải pháp hiện đại, linh hoạt và tiết kiệm chi phí, giúp các ứng dụng mở rộng tự động mà không cần quản lý máy chủ.
- Tuy nhiên, nó không phù hợp cho các ứng dụng cần hiệu suất ổn định hoặc xử lý lâu dài.
- Serverless đang trở thành xu hướng chính trong phát triển phần mềm hiện đại, đặc biệt là các hệ thống cloud-native.

Chúc các bạn học tốt
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN TP.HCM

Nhóm UIT-Together
TS. Nguyễn Tấn Trần Minh Khang